LEADERS UNIVERSITY / LICENCE BI

# Tronc commun : Solen Commerce 360, séances 1 à 4

IA, Codex et BI · le livret central du binôme · de la demande de la direction à un entrepôt historisé, contrôlé et relu à deux

| 4 | 4 | 0 |
| --- | --- | --- |
| SÉANCES DE 4 H (S1 → S4) | SOURCES : CSV, EXCEL, JSON, SQL | LIGNE DE CODE À LA MAIN |

**SOLEN DIGITAL · CLIENT FICTIF MAISON ZITOUNA**

Échantillon → rejets → étoile → lot 2 et SCD2 → revue et CI

02 / TRONC COMMUN

Youssef Douma · Leaders University

Édition pédagogique · 28 octobre 2026 · données fictives

02 / TRONC COMMUN · SOMMAIRE

# Sommaire

Séances 1 à 4, mission d'entraînement **Solen Commerce 360** (client fictif Maison Zitouna). Chaque séance s'ouvre sur un épisode Solen, puis des missions numérotées (S1-M1, S1-M2…).


| Partie | Contenu | Page |
|---|---|---|
| **Avant de commencer** | Comment lire ce livret · la mission en une page | 3 |
| **Séance 1 · Cadrage** | Vérification des postes et GitHub à deux (livret 01, D–E : PR n°1 de A), le mail de la direction, l'échantillon à la main, les outils, la première demande à Codex, la PR n°2 (B auteur) | 5 |
| **Séance 2 · Premier résultat… faux** | pandas (CSV, Excel, JSON), fonctions, modules, exceptions, règles de nettoyage et rejets, une modification demandée à Codex avec son test, PR A → revue B | 17 |
| **Séance 3 · Base opérationnelle et étoile** | SQL paramétré, grain, étoile, clés, classe Entrepot, chargement initial, totaux, choix de la mission, PR B → revue A | 30 |
| **Séance 4 · Le besoin change** | lot 2, SCD2, rejeu, journal des chargements, tests et CI, revue → correction → revue, conflit contrôlé, export, passer à votre mission ; extension : un Skill local | 42 |
| **Annexes** | Les nombres de référence · lire un ÉCHEC | 58 |


## Six règles pour tout le livret

- **Prévoir avant de demander** : écrivez la valeur attendue avant d'envoyer.
- **Codex écrit le code, vous contrôlez** : vous tapez des commandes, jamais de Python, SQL ou YAML.
- **Une demande = une petite étape** avec un résultat observable.
- **Relancer vous-même** une commande « lancée » par Codex : c'est votre preuve.
- **Ne jamais toucher** aux sources, à `reference/` ni à `controles/`.
- <chevrons> = à remplacer en entier : <branche> devient `lecture-nettoyage`.

### Les autres livrets

`01_demarrer.pdf` : parties A–C chez vous avant la séance 1 (séance 0), parties D–E en début de séance 1 (GitHub à deux, PR n°1). `03_diriger_codex_et_git.pdf` : chaque fois que vous ouvrez une branche ou une PR. `04_catalogue_missions.pdf` : fin de séance 3 pour choisir votre mission, fin de séance 4 pour passer à votre mission. `05_livrer_soutenir.pdf` : séance 5. `06_aide_memoire.pdf` : à garder ouvert.

02 / TRONC COMMUN · AVANT DE COMMENCER

# Comment lire ce livret

Chaque mission suit le même chemin : **quoi** (le résultat visé), **pourquoi** (la question de la direction qu'il sert), **la notion** (un schéma ou un petit tableau), **l'action exacte** (où cliquer, quoi taper), **la demande à Codex**, **le résultat attendu** avec les vrais nombres de la mission, **le contrôle**, les **erreurs courantes**, et ce que vous savez faire à la fin.

| Bloc | Ce qu'il contient | Ce que vous faites |
|---|---|---|
| **Où :** | Le lieu de l'action : terminal Ubuntu, application Codex, ou GitHub dans le navigateur. | Ouvrez le bon outil avant de lire la suite. |
| Demande à Codex | Un texte prêt à adapter, tiré du dossier `prompts/` du cours. | Remplacez chaque <…>, relisez, puis envoyez dans l'application Codex. |
| Terminal Ubuntu (bash) | Une commande que **vous** tapez (ou collez) dans le terminal, dans `~/solen-bi`, venv actif. | Tapez-la, Entrée, lisez la sortie. |
| Sortie attendue | Ce que l'écran devrait afficher. Les nombres viennent de la référence de la mission. | Comparez ; ne la tapez jamais. |
| Exemple de code produit par Codex | Un court extrait du genre de code que Codex écrit. Le vôtre sera différent. | Lisez-le, expliquez-le ; ne le tapez pas. |
| Contrôle | La commande qui tranche (`python -m controles.verifier …`) et ce qu'affiche une réussite (`OK`) ou un échec (`ÉCHEC`). | Lancez-la vous-même. |
| Erreurs courantes | Deux ou trois pièges vus souvent, et la sortie. | Lisez-les **avant** d'être bloqué. |
| À ce stade, je sais… | Une courte liste à cocher. | Cochez seulement ce que vous savez montrer à votre binôme. |

## Les durées

Les minutages de chaque séance sont des **estimations** de conception, à confirmer par un essai réel. Avancez au rythme du binôme : une mission comprise vaut mieux que deux missions recopiées.

02 / TRONC COMMUN · AVANT DE COMMENCER

# La mission en une page

Maison Zitouna vend huile d'olive, olives, dattes, savons et coffrets dans **trois boutiques** : Tunis Centre (`TUN`, zone Grand Tunis), Nabeul (`NBE`, zone Cap Bon) et Sfax (`SFX`, zone Sud). Quatre sources décrivent son activité de 2026.

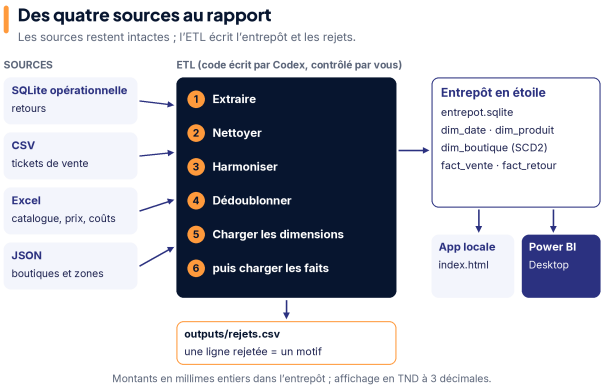



Schéma · Les quatre sources passent par l'ETL écrit par Codex : extraire, nettoyer, harmoniser, dédoublonner, charger les dimensions puis les faits. Les lignes écartées vont dans `outputs/rejets.csv` avec leur motif.

| Source (dans `tronc-commun/sources/lot1/`) | Contenu | Taille lot 1 |
|---|---|---|
| `ventes.csv` | Lignes de tickets de caisse, séparateur `;` | 1998 lignes + en-tête |
| `catalogue.xlsx` | Feuilles `Catalogue` (en-tête ligne 3) et `Familles` | 8 produits, 6 familles |
| `boutiques.json` | Boutiques et historique de zone | 3 boutiques |
| `operations.sqlite` | Base des retours : tables `retour`, `motif_retour`, `parametre` | 82 retours (78 VALIDE, 4 ANNULE) |

| Indicateur | Définition exacte (cahier des charges) |
|---|---|
| Ventes brutes | somme de quantité × prix unitaire du ticket, lignes acceptées |
| Retours | somme des montants remboursés des retours VALIDE acceptés |
| Ventes nettes | ventes brutes − retours |
| Marge sur coût d'achat | ventes nettes − coût d'achat des unités nettes |
| Taux de retour | retours / ventes brutes (en montant) |

Montants stockés en **millimes entiers** (1 TND = 1 000 millimes), affichés en TND à 3 décimales. « Marge » veut toujours dire **marge sur coût d'achat**, jamais « bénéfice ».

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# Séance 1 · Lundi, 9 h : le mail de la direction

Premier jour chez Solen Digital. Karim Jaziri, responsable BI (personnage fictif), vous fait asseoir, ouvre sa messagerie et vous transfère ce mail, reçu vendredi soir :

### Épisode 1 · Mail de Lamia Ben Salah, directrice (fictive)

« Bonjour à tous les deux,

Les ventes montent, mais où passe la marge ? Je veux, par boutique, par zone et par mois : les ventes brutes, les retours, les ventes nettes, la marge sur coût d'achat et le taux de retour. Et je veux pouvoir faire confiance aux chiffres : dites-moi aussi ce que vous avez dû écarter, et pourquoi.

Pour information : à partir du 15 juillet, la boutique de Nabeul passe de la zone Cap Bon à la zone Nord-Est. Les chiffres du premier semestre doivent rester dans Cap Bon. »

## Minutage de la séance 1 (240 min, estimations)

| Min | Format | Contenu |
|---|---|---|
| 30 | Pratique | Vérification des postes (livret 01, « Fin de la séance 0 ») |
| 10 + 45 | Expl. + pratique | GitHub à deux (livret 01, D–E) : dépôt, protection, venv, PR n°1 de A |
| 10 | Pause | — |
| 10 | Explication | Épisode 1, la mission, les rôles A et B |
| 10 + 10 | Expl. + pratique | S1-M1 · Traduire la demande en indicateurs |
| 5 + 30 | Expl. + pratique | S1-M2 · L'échantillon, à la main d'abord |
| 10 | Pause | — |
| 8 | Explication | Épisode 2 · Réunion de cadrage avec Karim |
| 10 + 20 | Expl. + pratique | S1-M3 · Les outils et le premier contrôle ; S1-M4 · Première demande à Codex |
| 5 + 27 | Expl. + pratique | S1-M5 · PR n°2 `cadrage-binome` (B auteur, A relecteur), puis note d'état |

Total : **58 min** d'explications, **162 min** de pratique, deux pauses de 10 min. Fin de séance : **deux PR relues et fusionnées** (n°1 de A, n°2 de B) ; votre calcul à la main = `echantillon/attendu.md`.

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# S1–M1 · Traduire la demande en indicateurs

**Quoi :** transformer le mail en une liste d'indicateurs définis, avec unité et grain. **Pourquoi :** une question vague (« où passe la marge ? ») ne se calcule pas ; un indicateur défini, si. **Où :** papier et `cahier_des_charges.md`.

| Mot | En une phrase | Exemple Maison Zitouna |
|---|---|---|
| **BI** (informatique décisionnelle) | Transformer des données en chiffres qui aident à décider. | « Sfax a 9 % de retours contre 2 % à Tunis : on regarde les coffrets. » |
| **ETL** | Extraire les sources, les Transformer (nettoyer), les Charger dans une base d'analyse. | Lire `ventes.csv`, écrire `18,500` en 18500 millimes, ranger la ligne dans `fact_vente`. |
| **Entrepôt** | Base faite pour analyser, séparée des sources. | `outputs/entrepot.sqlite`, recréé par le pipeline. |
| **Grain** | Ce que représente **une** ligne de la table de faits. | Une ligne de ticket : `TK10001-2` = 4 bouteilles de P01 à 18,500 TND. |

**Action, sur papier :** lisez le § 9 du cahier des charges, puis comparez à deux : la formule doit être **identique** au cahier des charges.

| Indicateur | Formule | Unité stockée | Unité affichée |
|---|---|---|---|
| Ventes brutes | Σ quantité × prix du ticket | millimes | TND, 3 décimales |
| Retours | Σ montant remboursé (VALIDE) | millimes | TND |
| Ventes nettes | brutes − retours | millimes | TND |
| Marge sur coût d'achat | nettes − coût d'achat des unités nettes | millimes | TND |
| Taux de retour | retours / brutes | ratio | %, 2 décimales |

## Erreurs courantes

• Écrire « bénéfice » : salaires et loyer ne sont pas déduits. On dit **marge sur coût d'achat**. Et le taux de retour se calcule **en montant**, pas en unités.

## À ce stade, je sais…

[ ] dire ce qu'est le grain, avec un exemple de ligne, et les formules des cinq indicateurs avec leur unité.

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# S1–M2 · L'échantillon, à la main d'abord

**Quoi :** décider, pour chacune des 12 lignes de `tronc-commun/echantillon/ventes.csv`, si elle est **gardée**, **ignorée** ou **rejetée**, puis calculer les indicateurs. **Pourquoi :** ces nombres serviront à vérifier le code de Codex. Si vous ne savez pas faire le calcul à la main, vous ne saurez pas dire si le code est faux. **Où :** papier et calculatrice. **Interdit :** demander à Codex avant d'avoir fini.

**Fichier tronc-commun/echantillon/ventes.csv · ligne 1 = en-tête · sans la colonne mode_paiement**

```
ligne_id;ticket_id;date_vente;boutique;produit;quantite;prix_unitaire
E-001-1;E-001;2026-03-02;TUN;P01;2;18.500
E-001-2;E-001;02/03/2026;TUN;P05;3;3,900
E-002-1;E-002;3 mars 2026; nbe ;P04;1;14.5
E-003-1;E-003;2026-03-05;SFX;P01;1;18,5
E-003-1;E-003;2026-03-05;SFX;P01;1;18,5
E-004-1;E-004;2026-03-06;Sfx;P04;2;14,500
E-005-1;E-005;2026-03-06;TUN;P01;1;16.650
E-006-1;E-006;2026-03-07;TUN;P05;2;3.900
E-006-1;E-006;2026-03-07;TUN;P05;4;3.900
;E-007;2026-03-08;NBE;P01;1;18.500
E-008-1;E-008;2026-03-09;TUN;P99;1;9.000
E-009-1;E-009;2026-03-09;NBE;P05;-1;3.900
```

## Les règles, dans l'ordre (cahier des charges § 4)

Le **premier** problème trouvé donne le motif. Un **rejet** est une ligne écartée et écrite dans `rejets.csv` avec son **motif** ; un **doublon exact** est simplement ignoré (on garde une occurrence).

| # | Test | Résultat |
|---|---|---|
| 1 | ligne identique, caractère pour caractère, à une ligne précédente | ignorée (doublon exact) |
| 2 | `ligne_id` vide | rejet `id_manquant` |
| 3 | même `ligne_id` sur plusieurs lignes **différentes** | rejet `id_conflit` pour **toutes** ces lignes |
| 4 | boutique ou produit absent des référentiels | rejet `cle_inconnue` |
| 5 | quantité non entière ou ≤ 0 | rejet `quantite_invalide` |

Normalisations (ce ne sont pas des rejets) : `18,5` = `18.500` = 18 500 millimes ; `02/03/2026` se lit **jour d'abord** (2 mars 2026) et `3 mars 2026` devient `2026-03-03` ; `nbe` et `Sfx` deviennent `NBE` et `SFX` (espaces retirés, majuscules).

Retours (cahier § 3 et § 4) : un retour **ANNULE** est ignoré ; un retour dont la ligne d'origine n'existe pas est rejeté en `cle_inconnue`.

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# S1-M2 · Calculer les cinq indicateurs

Sur papier, **sans regarder la page suivante**. Prix du **ticket** : `16.650` (E-005-1) est une promotion, on le garde.

| Ligne | ligne_id | Décision | Calcul | Brut (TND) |
|---|---|---|---|---|
| 2 | E-001-1 | gardée | 2 × 18,500 | 37,000 |
| 3 | E-001-2 | … | … | … |
| 4 | E-002-1 | … | … | … |
| 5 et 6 | E-003-1 | … | … | … |
| 7 | E-004-1 | … | … | … |
| 8 | E-005-1 | … | … | … |
| 9 à 13 | les 5 autres | motif ? | — | — |

| Retour | Ligne d'origine | Statut | Montant | Décision |
|---|---|---|---|---|
| R-0001 | E-004-1 (1 × dattes P04) | VALIDE | 14,5 | … |
| R-0002 | E-001-1 | ANNULE | 18,5 | … |
| R-0003 | E-099-9 | VALIDE | 18,5 | … |

Coût d'achat unitaire : P01 = 11,200 ; P04 = 8,750 ; P05 = 1,650 TND. Coût des **unités nettes** = coût des unités vendues − coût des unités retournées.

| Indicateur | Votre calcul | Votre résultat |
|---|---|---|
| Ventes brutes | | |
| Retours | | |
| Ventes nettes | | |
| Coût d'achat des unités nettes | | |
| Marge sur coût d'achat | | |
| Taux de retour | | |

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# S1–M2 · Comparer avec attendu.md

**Contrôle :** ouvrez `tronc-commun/echantillon/attendu.md`. Comparez case par case. Un écart n'est pas une honte : c'est exactement le genre d'écart que vous chercherez plus tard dans le code de Codex. Trouvez **la** ligne qui diffère et la règle mal appliquée.

| Résultat de référence | Valeur | Comment on l'obtient |
|---|---|---|
| Lignes | 12 lues → 6 gardées | 1 doublon exact ignoré ; 5 rejets : 2 `id_conflit`, 1 `id_manquant`, 1 `cle_inconnue` (P99), 1 `quantite_invalide` |
| Ventes brutes | **127,350 TND** | 37,000 + 11,700 + 14,500 + 18,500 + 29,000 + 16,650 (127350 millimes) |
| Retours | **14,500 TND** | R-0001 gardé ; R-0002 ANNULE ignoré ; R-0003 rejet `cle_inconnue` (E-099-9 n'existe pas) |
| Ventes nettes | **112,850 TND** | 127,350 − 14,500 |
| Coût des unités nettes | **67,250 TND** | 76,000 vendus − 8,750 retournés |
| Marge sur coût d'achat | **45,600 TND** | 112,850 − 67,250 |
| Taux de retour | **11,39 %** | 14500 / 127350 |

Attendu aussi pour l'entrepôt de l'échantillon : `fact_vente` 6 lignes, `fact_retour` 1, `dim_produit` 8, `dim_boutique` 3, `dim_date` 31 jours (mars 2026), 6 lignes de rejets (5 ventes + 1 retour).

## Erreurs courantes

- Compter le doublon exact E-003-1 comme un `id_conflit` : les deux lignes sont **identiques**, on en garde une. `id_conflit`, c'est même identifiant avec un contenu **différent** (E-006-1 : quantité 2 puis 4).
- Lire `02/03/2026` comme le 3 février : le format `JJ/MM/AAAA` met le **jour** d'abord.
- Compter R-0002 comme un rejet : un retour ANNULE est **ignoré**, ce n'est pas une erreur de données.

## À ce stade, je sais…

[ ] décider à la main du sort d'une ligne et citer son motif ;
[ ] calculer ventes nettes, marge et taux de retour de l'échantillon, avec l'unité ;
[ ] expliquer pourquoi un doublon exact n'est pas un rejet.

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# Épisode 2 · La réunion de cadrage

Après la pause, Karim Jaziri réunit le binôme dix minutes. Il veut que chaque définition soit **tranchée et écrite** avant la première ligne de code. Voici ce qui est décidé. Recopiez les décisions dans `notes/JOURNAL.md` (rubrique **Décision**) pendant S1-M5.

| Question posée | Décision | Conséquence sur un chiffre |
|---|---|---|
| Que représente une ligne de fait ? | **Une ligne de ticket** (grain). | Un ticket de 3 produits = 3 lignes dans `fact_vente`. |
| Quel prix pour les ventes brutes ? | Le prix **du ticket**, pas celui du catalogue. | E-005-1 compte 16,650 TND, pas 18,500. |
| Un retour ANNULE compte-t-il ? | Non : ignoré, pas rejeté. | R-0002 (18,5) ne réduit pas les ventes nettes. |
| Que veut dire « marge » ? | Marge **sur coût d'achat** = nettes − coût d'achat des unités nettes. | Échantillon : 45,600 TND. |
| Comment stocker l'argent ? | En **millimes entiers**. | 18,500 TND → 18500 ; jamais 18.499999. |
| Un retour du 20 juillet d'un achat du 30 juin à Nabeul : quelle zone ? | **B** : la zone valable à la date **du retour** (Nord-Est), compté dans **son** mois. | Le retour pèse sur Nord-Est en juillet, pas sur Cap Bon en juin. |

## Pourquoi des millimes entiers ?

L'ordinateur stocke mal certains décimaux : `0.1 + 0.2` ne vaut pas exactement `0.3` en Python. Sur 2 000 lignes, ces poussières font varier le total d'un millime. En entiers, 18500 + 3900 = 22400, toujours. On ne repasse en TND (÷ 1 000) qu'au moment d'afficher.

### La dernière règle, la plus importante

Ces décisions sont **les vôtres** et celles de la direction. Codex peut proposer une question à trancher ; il ne la tranche pas. Si Codex propose une autre définition, la référence est le cahier des charges.

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# S1–M3 · Vos outils : qui fait quoi, où

**Quoi :** savoir dans quel outil se fait chaque geste. **Pourquoi :** la moitié des blocages du premier jour viennent d'une commande tapée au mauvais endroit. **Prérequis :** livret 01 fait : parties A–C en séance 0, D–E en début de séance (dépôt `~/solen-bi` cloné par A et B, venv créé, PR n°1 fusionnée).

| Outil | Ce que vous y faites | Exemple |
|---|---|---|
| Application Codex | Écrire une demande, lire la réponse et le diff, accepter ou refuser. | Projet `solen-bi`, modèle **GPT-6 Luna**, effort **High** pour commencer. |
| Terminal Ubuntu | Lancer les commandes : Git, Python, contrôles. | `python -m controles.verifier --mission tronc-commun --etape initial` |
| GitHub (navigateur) | Pull requests, revues écrites, CI (onglet **Actions**), fusion. | **Files changed → Review changes → Approve** |
| Papier | Prévoir une valeur, calculer l'échantillon, dessiner l'étoile. | « J'attends 1960 lignes de faits. » |




Schéma · Trois emplacements : votre copie locale, celle de votre binôme, et GitHub. `commit` enregistre chez vous, `push` envoie, `pull` récupère, la pull request propose de fusionner une branche dans `main`.

## Effort, contexte, quota : trois mots à ne pas confondre

| Mot | Ce que c'est | Ce que vous faites |
|---|---|---|
| Effort | Le temps de réflexion choisi : Light, Medium, High, Extra High. « Standard » est une vitesse, pas un effort. | High par défaut ; Medium pour une petite fonction bien délimitée. |
| Contexte | Ce que la conversation contient (fichiers lus, échanges). | Une conversation par étape ; une neuve si elle devient longue. |
| Quota | La limite d'usage du compte, affichée dans l'application. | Quota atteint : continuez les gestes locaux (contrôles, diff, calcul). Aucun achat. |

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# S1–M3 · Un premier échec, attendu

**Quoi :** lancer les contrôles fournis sur un dépôt où rien n'est construit. **Pourquoi :** partir d'un échec utile ; chaque PR en fera passer un morceau au vert. **Où :** terminal Ubuntu, chacun sur son poste. **Prévoir :** réussite ou échec, pour chacune des deux commandes ?

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
cd ~/solen-bi
source .venv/bin/activate
cat mission.txt
python -m controles.verifier --mission tronc-commun --etape initial
pytest controles
```

**Sortie attendue · ne pas recopier**

```
tronc-commun

== Étape initial : comparaison avec attendu_lot1.json ==
ÉCHEC — entrepôt absent : attendu /home/<vous>/solen-bi/outputs/entrepot.sqlite

Résultat : 0 OK, 1 ÉCHEC.
...
controles/test_contrat.py ...sss...
6 passed, 3 skipped
```

• Le vérificateur **compare vos sorties** (`outputs/`) à la référence ; il ne lit jamais votre code. Chaque ligne commence par `OK`, `ÉCHEC`, `INFO` ou `À FAIRE` ; la dernière fait le bilan. Ici l'échec est normal : `solen/` est vide.

• `pytest controles` : 6 tests (référence, sources) **passent** ; les 3 tests sur vos sorties sont **ignorés** (s = skipped) tant que les sorties n'existent pas.

• `echo $?` juste après le vérificateur affiche le code de sortie : `1` = au moins un échec. La CI s'en sert.

## Erreurs courantes

• `No module named controles` : pas à la racine du dépôt (`cd ~/solen-bi`). `No module named pandas` : le venv n'est pas actif, l'invite doit commencer par `(.venv)`.
• Modifier `controles/` ou `reference/` pour « voir du vert » : interdit, et visible dans le diff.

## À ce stade, je sais…

[ ] dire où se fait chaque geste (Codex, terminal, GitHub) ;
[ ] lancer le vérificateur et lire la première ligne `ÉCHEC`.

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# S1–M4 · Première demande à Codex : le cadrage

**Quoi :** faire relire votre cadrage par Codex, **sans code**. **Pourquoi :** apprendre à écrire une demande précise, et à juger une réponse avec votre propre calcul. **Où :** application Codex, projet `solen-bi`, modèle GPT-6 Luna, effort High. **Avant :** vous avez votre total des ventes brutes de l'échantillon (S1-M2).

**Demande à Codex · texte à adapter**

```
Lis mission.txt, tronc-commun/cahier_des_charges.md,
tronc-commun/dictionnaire.md et les fichiers de tronc-commun/echantillon/.
Ne modifie aucun fichier et n'écris pas de code.
Pour chaque indicateur demandé par la direction, donne en tableau :
nom, définition en une phrase, formule, unité (millimes, TND, %, quantité),
grain du fait utilisé, et le calcul sur l'échantillon ligne par ligne.
Mon calcul à la main des ventes brutes donne <ma valeur> TND :
compare et explique l'écart s'il y en a un.
Liste ensuite au plus 3 questions ambiguës à poser à la direction,
avec la conséquence de chaque réponse sur un chiffre.
```

## Résultat attendu

- Cinq indicateurs, chacun avec une unité, une formule **identique** au cahier des charges : ventes brutes 127,350 TND ; retours 14,500 ; nettes 112,850 ; marge sur coût d'achat 45,600 ; taux de retour 11,39 %.
- Le calcul ligne par ligne retombe sur `echantillon/attendu.md`. S'il diffère de votre calcul, l'un des deux a tort : retrouvez la ligne en cause, à deux.
- Des questions **métier** (exemple : « un retour plus de 30 jours après la vente est-il accepté ? »), pas des questions techniques.

## Contrôle

Codex ne devait rien modifier. Dans le terminal : `git status` doit afficher `nothing to commit, working tree clean`. Sinon, demandez : « Qu'as-tu modifié ? Annule ces changements. » et vérifiez de nouveau.

## Erreurs courantes

- Accepter la valeur de Codex sans avoir fait votre propre calcul : vous ne saurez pas l'expliquer.
- Mélanger millimes et TND : 12500 « TND » au lieu de 12,500 TND.
- Laisser un <…> dans la demande : elle n'est pas prête.

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# S1–M5 · PR n°2 : B auteur, la branche

**Quoi :** la **PR n°2** du binôme : présenter le binôme dans `README.md` et ajouter au journal une entrée « PR #2 » avec les décisions de cadrage. **B** est auteur, **A** relit : les rôles s'inversent après la PR n°1 (livret 01, cycle 1). **Pourquoi :** chacun doit avoir fait le cycle complet comme auteur, sur un changement sans risque, avant le premier code.

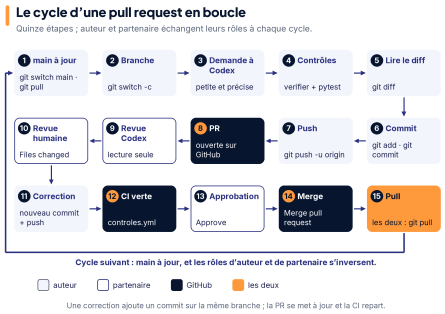



Schéma · Le cycle d'une PR : main à jour, branche, demande, contrôles, diff, commit, push, PR, revue Codex en lecture seule, revue humaine, correction, CI verte, approbation, merge, pull par les deux. Puis les rôles s'inversent.

## B · Terminal Ubuntu : partir d'un main à jour

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git switch main
git pull
git switch -c cadrage-binome
```

**Sortie attendue · ne pas recopier**

```
Switched to a new branch 'cadrage-binome'
```

**Demande à Codex · texte à adapter**

```
Sur la branche cadrage-binome, prépare deux changements de texte, sans code :
1. README.md : ajoute en haut une section « Notre binôme » avec
   A = <prénom A> (<compte GitHub A>), B = <prénom B> (<compte GitHub B>),
   mission en cours : tronc-commun ;
2. notes/JOURNAL.md : ajoute en haut une entrée « PR #2 — Cadrage »
   (copie le modèle en commentaire) : date du jour, auteur B, relecteur A,
   nos décisions de cadrage (grain, prix du ticket, millimes, règle B)
   et la preuve : la ligne ÉCHEC du vérificateur, recopiée telle quelle :
   <coller la ligne>.
Ne modifie aucun autre fichier. Ne fais ni commit ni push.
```

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# S1–M5 · PR n°2 : la revue de A

**Markdown** (`.md`) : du texte avec quelques signes, # pour un titre, - pour une puce. Ce n'est pas du code : écrivez ces textes vous-mêmes (nano, livret 01) ou faites faire un brouillon à Codex, puis corrigez-le.

## B · Lire le diff, puis enregistrer et envoyer

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git status
git diff
git add README.md notes/JOURNAL.md
git commit -m "Cadrage : binôme, décisions et premier contrôle"
git push -u origin cadrage-binome
```

`git status` doit lister **deux** fichiers modifiés, rien d'autre (ni `outputs/`, ni `.venv/`). `git diff` montre les lignes : + ajoutée, - retirée. Lisez-les toutes avant `git add` ; on ajoute les fichiers **par leur nom**.

## B · GitHub : ouvrir la PR

- **Compare & pull request** ; vérifiez **base: main ← compare: cadrage-binome**. Titre : celui du commit.
- Description : ce qui change, la preuve (`ÉCHEC — entrepôt absent`), à relire en priorité. **Reviewers** : le compte de A. **Create pull request**.

## A · Relire et décider

- Onglet **Files changed** : lisez chaque ligne. Une décision du journal est-elle fausse ou floue ? Cliquez sur la ligne et écrivez un commentaire précis.
- **Review changes** → **Approve** si tout est juste, **Request changes** sinon. Exemple utile : « La règle B est mal recopiée : un retour compte dans la zone du jour du **retour**. »
- Onglet **Checks** : la CI « Contrôles Solen » tourne. Avec `ETAPE_CI:` aucune, elle vérifie seulement l'installation : elle doit être verte.
- CI verte + approbation : **A** clique **Merge pull request**. L'auteur ne fusionne jamais sa propre PR.

**A et B · Revenir à main**

```
git switch main
git pull
git log --oneline -3     # en haut : la fusion de la PR #2
```

**Erreurs courantes** — Commit fait sur `main` par erreur : ne poussez pas ; voir « Erreurs Git courantes » (livret 01). Jamais de commande « forcée » sans le professeur. · A approuve sans lire : une revue utile cite au moins une ligne précise.

02 / TRONC COMMUN · SÉANCE 1 · CADRAGE

# Fin de séance 1 · Note d'état et bilan

Dernières minutes : une **note d'état** courte vous permet de reprendre en séance 2 sans relire toute la conversation. **Où :** application Codex, puis `notes/JOURNAL.md` si nécessaire.

**Demande à Codex · texte à adapter**

```
Rédige une note d'état de 8 lignes maximum, que je copierai moi-même.
Ne modifie aucun fichier.
Contenu : mission (mission.txt), branche actuelle, ce qui fonctionne,
commandes réellement exécutées et leur résultat, défaut encore ouvert,
prochaine petite étape et sa preuve de réussite.
Sépare clairement : réalisé / à faire / non vérifié.
Aucun identifiant de compte.
```

Vérifiez **une** ligne de la note contre le terminal. Une note qui affirme « vérificateur OK » est fausse : rien n'est encore construit.

## Preuves de la séance 1

| Preuve | Où la voir |
|---|---|
| Calcul à la main = `attendu.md` (127,350 / 112,850 / 45,600 / 11,39 %) | votre feuille, et l'entrée PR #2 du journal |
| Définitions des cinq indicateurs avec unité | journal, rubrique Décision |
| Premier contrôle : `ÉCHEC — entrepôt absent` | terminal ; recopié dans le journal |
| PR n°1 (A) et PR n°2 (B) relues et fusionnées, CI verte | GitHub, onglet Pull requests (Closed) |
| `main` à jour chez A et B | `git log --oneline -3` sur les deux postes |

### À ce stade, je sais…

[ ] expliquer la question de la direction et les cinq indicateurs ;
[ ] calculer l'échantillon à la main et justifier chaque rejet ;
[ ] faire le cycle complet de PR, comme auteur et comme relecteur.

### Avant la séance 2 (≈ 20 min)

Relisez le § 3 du cahier des charges (règles de nettoyage) et ouvrez `tronc-commun/sources/lot1/ventes.csv` : repérez à l'œil un prix écrit avec une virgule, un autre avec un point, et une date en toutes lettres.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# Séance 2 · Un premier résultat… faux

Mardi matin. Karim Jaziri arrive avec un sourire et une feuille imprimée.

### Épisode 3 · Ce que Karim a calculé

« J'ai additionné quantité × prix sur tout `ventes.csv` du lot 1 dans un tableur : 1998 lignes, **78 839,780 TND** de ventes brutes. C'est cohérent avec ce que la direction imagine. Lamia veut l'envoyer au client cet après-midi. Vous confirmez ? »

Vous refaites sa méthode sur l'échantillon : toutes les lignes, sans aucune règle. Résultat : **192,850 TND**, alors que votre calcul à la main de la séance 1 donne **127,350 TND**. Le doublon compte deux fois, les lignes en conflit, sans identifiant, au produit inconnu et à quantité négative sont toutes additionnées. Sur le lot 1, le vrai total après nettoyage sera **78 067,830 TND** : 771,950 TND de moins. Un chiffre plausible n'est pas un chiffre juste.

## Minutage de la séance 2 (240 min, estimations)

| Min | Format | Contenu |
|---|---|---|
| 12 | Explication | Épisode 3 : pourquoi le total de Karim est faux |
| 14 + 30 | Expl. + pratique | S2-M1 · Lire CSV, Excel et JSON avec pandas |
| 10 | Pause | — |
| 14 + 40 | Expl. + pratique | S2-M2 · Nettoyer et rejeter avec un motif |
| 14 + 25 | Expl. + pratique | S2-M3 · Fonctions, modules, exceptions ; une modification demandée |
| 10 | Pause | — |
| 12 + 49 | Expl. + pratique | S2-M4 · PR : A auteur, B relecteur, correction |
| 10 | Pratique | Note d'état, journal |

Total : **66 min** d'explications, **154 min** de pratique, deux pauses. Preuve en fin de séance : `outputs/rejets.csv` existe, les comptes gardés / rejetés de `ventes.csv` sont égaux à la référence, et la PR de A est relue par B, corrigée si besoin, fusionnée.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2–M1 · Lire trois formats avec pandas

**Quoi :** lire `ventes.csv`, `catalogue.xlsx` et `boutiques.json` du lot 1 **sans rien corriger**. **Pourquoi :** si la lecture abîme une valeur (`18,500` lu comme 18,5 € ou comme du texte vide), aucun nettoyage ne la rattrapera. **pandas** est la bibliothèque Python qui lit ces fichiers ; elle les range dans un **DataFrame**, un tableau en mémoire (lignes numérotées, colonnes nommées). Le fichier, lui, reste intact sur le disque.

| Source | Piège | Ce que la lecture doit faire |
|---|---|---|
| `ventes.csv` | séparateur `;` ; prix `18,500`, `18.5`, `1 240,000` ; trois formats de date | séparateur `;`, encodage UTF-8, **toutes les colonnes en texte** : on convertira au nettoyage |
| `catalogue.xlsx` | un titre au-dessus du tableau ; deux feuilles | feuille `Catalogue`, en-tête à la **ligne 3** (pandas compte à partir de 0 : `header=2`) ; feuille `Familles` à part |
| `boutiques.json` | liste imbriquée : chaque boutique a un `historique` | parcourir les boutiques puis leurs versions (3 au lot 1) |
| Encodage | espèces, chèque | UTF-8 ; Ã¨ = mauvais encodage (livret 01) |

## Prévoir, dans le terminal

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
wc -l tronc-commun/sources/lot1/ventes.csv
grep -c '"code"' tronc-commun/sources/lot1/boutiques.json
```

**Sortie attendue · ne pas recopier**

```
1999 tronc-commun/sources/lot1/ventes.csv
3
```

`wc -l` compte 1999 lignes : **1998 lignes de données** + l'en-tête. Notez vos prévisions : 1998 lignes CSV, 8 produits, 3 boutiques.

## Erreurs courantes

• Laisser pandas convertir les prix dès la lecture : `18,500` devient du texte et `18.500` devient 18.5, deux traitements pour la même valeur.
• Catalogue lu avec la ligne de titre comme en-tête : colonnes `Unnamed: 1`, `Unnamed: 2`… : l'en-tête est à la ligne 3.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2–M1 · La demande de lecture

**Où : A** au clavier, **B** à côté qui lit et pose des questions. Terminal Ubuntu, puis application Codex.

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git switch main
git pull
git switch -c lecture-nettoyage
```

**Demande à Codex · texte à adapter**

```
Lis mission.txt et tronc-commun/dictionnaire.md. Crée dans solen/ un module
de lecture des sources de tronc-commun/sources/lot1/ : ventes.csv,
catalogue.xlsx (feuilles Catalogue et Familles) et boutiques.json,
avec pandas, toutes les colonnes en texte, sans nettoyer.
Bon séparateur, bon encodage, en-tête du catalogue à la ligne 3.
Ajoute une commande temporaire python -m solen sources qui affiche pour
chaque source : nom, nombre de lignes, colonnes, 3 premières lignes.
Lance-la et recopie la sortie. J'attends 1998 lignes CSV, 8 produits,
6 familles, 3 boutiques : compare. Explique en 4 lignes un DataFrame et
une exception possible à la lecture. Ne fais ni commit ni push.
```

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
python -m solen sources
```

**Sortie attendue · forme libre, les nombres comptent**

```
ventes.csv : 1998 lignes, 8 colonnes (ligne_id, ticket_id, date_vente, ...)
catalogue.xlsx / Catalogue : 8 lignes · Familles : 6 lignes
boutiques.json : 3 boutiques
```

## Contrôle

Vos trois nombres sont retrouvés. Les accents s'affichent (chèque, pas chÃ¨que). `git diff` ne touche que `solen/` ; aucune ligne n'est déjà corrigée ni supprimée. Dans le diff, cherchez `dtype=str` (ou équivalent) : la preuve que tout est lu en texte.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2-M2 · Les règles de nettoyage

**Quoi :** appliquer les règles du cahier des charges (§ 3 et § 4) et écrire chaque ligne écartée dans `outputs/rejets.csv` avec son **motif**. **Pourquoi :** la direction veut savoir « ce que vous avez dû écarter, et pourquoi ». Supprimer sans trace, c'est demander à être cru sur parole.

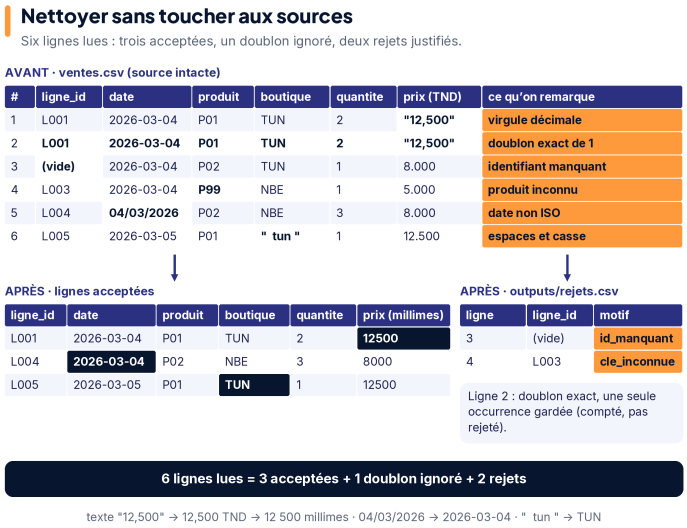



Schéma · Avant / après sur six lignes : formats normalisés (millimes entiers, dates ISO, codes en majuscules), doublon exact ignoré, deux rejets écrits avec leur motif. Les sources restent intactes.

## Ce que contient vraiment le lot 1 (référence `attendu_lot1.json`)

| Cas | Exemple réel dans `ventes.csv` | Lot 1 |
|---|---|---|
| Doublon exact (ignoré) | deux lignes identiques caractère pour caractère | 14 lignes |
| `id_conflit` | `TK10111-9` aux lignes 23 (qté 2) et 858 (qté 1) | 6 lignes |
| `id_manquant` | ligne 445 : `ligne_id` vide | 5 lignes |
| `cle_inconnue` | ligne 699 : boutique `BZT` ; produits `P09`, `P13` | 6 lignes |
| `quantite_invalide` | ligne 762 : `-2` ; ligne 1015 : `1.5` | 7 lignes |
| Gardées | 1998 − 14 − 24 | **1960** |

À normaliser sans rejeter (rubrique `nettoyage`) : 597 montants, 496 dates et 208 codes (casse, espaces) écrits autrement que le format cible.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2-M2 · La demande de nettoyage

**Où :** application Codex, même branche `lecture-nettoyage`. **Avant :** gardez votre feuille de la séance 1 : vos comptes sur l'échantillon sont la première preuve.

**Demande à Codex · texte à adapter**

```
Applique les règles des § 3 et § 4 de tronc-commun/cahier_des_charges.md
aux lignes de ventes.csv, dans solen/ seulement. Sources intactes.
Dans l'ordre : doublon exact = garder une occurrence (pas un rejet) ;
ligne_id vide = id_manquant ; même ligne_id sur des lignes différentes =
rejeter toutes ces lignes, id_conflit ; boutique ou produit inconnus =
cle_inconnue ; quantité <= 0 ou non entière = quantite_invalide.
Prix en millimes entiers (18,500 / 18.5 / 1 240,000), dates AAAA-MM-JJ
(JJ/MM/AAAA = jour d'abord), codes sans espaces et en majuscules.
Écris outputs/rejets.csv : colonnes lot, source, numero_ligne, motif
(l'en-tête du fichier source est la ligne 1).
Pour l'instant, python -m solen initial fait seulement cela et affiche
les comptes ; l'entrepôt viendra à la séance 3.
D'abord python -m solen echantillon, même traitement sur l'échantillon :
j'attends 6 gardées, id_conflit 2, id_manquant 1, cle_inconnue 1,
quantite_invalide 1. Puis lot1 : affiche aussi les ventes brutes des
lignes gardées en millimes.
Montre la fonction qui décide du motif. Ne fais ni commit ni push.
```

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
python -m solen echantillon
python -m solen initial
```

**Sortie attendue · forme libre, les nombres comptent**

```
echantillon · 12 lues, 1 doublon exact ignoré, gardées 6, rejetées 5
lot1 · ventes.csv : 1998 lignes lues, 14 doublons exacts ignorés
gardées 1960 · rejetées 24 : cle_inconnue 6, id_conflit 6,
id_manquant 5, quantite_invalide 7
ventes brutes des lignes gardées : 78067830 millimes (78 067,830 TND)
```

## Erreurs courantes

- Un compte diffère d'une unité : faites-vous montrer **la** ligne en cause (numéro de ligne dans `rejets.csv`), puis relisez la règle dans le cahier des charges avant de demander une correction.
- Le doublon exact rejeté en `id_conflit` : l'ordre des règles est faux (le doublon exact se traite en premier).
- `12.5 * 1000` calculé en flottant : exigez un calcul en entiers.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2-M2 · Contrôler les rejets

**Contrôle :** l'entrepôt n'existe pas encore, mais le vérificateur juge déjà les rejets de `ventes.csv` : `--etape rejets` compare `outputs/rejets.csv` à la référence, motif par motif, pour cette seule source. Attendu : **0 ÉCHEC**.

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
head -3 outputs/rejets.csv
python -m controles.verifier --mission tronc-commun --etape rejets
```

**Sortie attendue · ne pas recopier**

```
lot,source,numero_ligne,identifiant,motif,detail
lot1,ventes.csv,23,TK10111-9,id_conflit,2 lignes différentes
lot1,ventes.csv,416,TK10074-9,id_conflit,2 lignes différentes
INFO — mission tronc-commun · référence …/reference · sorties …/outputs

== Étape rejets : ventes.csv (lot1) comparé à attendu_lot1.json ==
OK — rejets lot1 · ventes.csv : cle_inconnue = 6
OK — rejets lot1 · ventes.csv : id_conflit = 6
OK — rejets lot1 · ventes.csv : id_manquant = 5
OK — rejets lot1 · ventes.csv : quantite_invalide = 7
OK — rejets lot1 · ventes.csv : 24 numero_ligne réels (2 à 1999), uniques
INFO — rejets de operations.sqlite et entrepôt : contrôlés par --etape initial

Résultat : 5 OK, 0 ÉCHEC.
```

À la séance 3, `--etape initial` contrôlera **toutes** les sources : **8** `cle_inconnue`, les 6 de `ventes.csv` + 2 retours de la base SQLite. Chaque `numero_ligne` doit désigner une vraie ligne du fichier (l'en-tête est la ligne 1). Et le total des ventes brutes, 78 067 830 millimes, est exactement l'indicateur `ventes_brutes_millimes` de la référence.

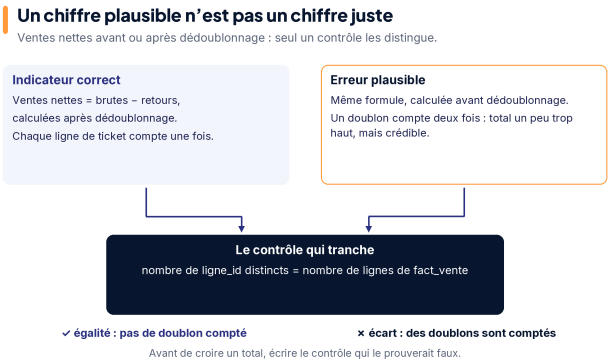



Schéma · Le même calcul avant et après dédoublonnage : seul un contrôle tranche. Ici : nombre de `ligne_id` distincts parmi les lignes gardées = nombre de lignes de faits.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2-M3 · Fonctions, modules, exceptions

**Quoi :** reconnaître dans le code de Codex trois notions de Python, sans en écrire une ligne. **Pourquoi :** à l'oral, on vous demandera « où la ligne est-elle rejetée ? » ; il faut savoir ouvrir le bon fichier et montrer la bonne ligne.

| Mot | Ce que c'est | Exemple chez vous |
|---|---|---|
| **Module** | Un fichier `.py` qui regroupe des fonctions d'un même thème. | `solen/lecture.py` |
| **Paquet** | Un dossier de modules. `python -m solen` lance `solen/__main__.py`. | `solen/` |
| **Fonction** | Un bloc nommé `def nom(...)` : reçoit des valeurs (paramètres), renvoie un résultat (`return`). | `def lire_csv_texte(chemin, sep=";")` |
| **Exception** | Un signal d'erreur lancé par `raise`, rattrapé plus loin par `try … except`. | `raise RejetLigne("quantite_invalide", …)` |
| **import** | Utiliser un module dans un autre. | `from .erreurs import RejetLigne` |

**Exemple de code produit par Codex · à lire, pas à taper**

```
# solen/lecture.py : un module, une fonction, une exception
import pandas as pd
from .erreurs import ErreurSource


def lire_csv_texte(chemin, sep=";", encodage="utf-8"):
    """Toutes les colonnes en texte : rien n'est converti."""
    if not chemin.exists():
        raise ErreurSource(f"source absente : {chemin}")
    return pd.read_csv(chemin, sep=sep, dtype=str,
                       keep_default_na=False, encoding=encodage)
```

Lisez à voix haute : « La fonction `lire_csv_texte` reçoit un chemin ; si le fichier n'existe pas, elle **lève** l'exception `ErreurSource` et le chargement s'arrête ; sinon elle **renvoie** un DataFrame où tout est texte. »

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2-M3 · Où est la fonction qui rejette une ligne ?

**Action :** chercher vous-mêmes, puis faire expliquer. `grep -n` affiche les lignes qui contiennent un mot, avec leur numéro. Les noms **et les fichiers** ci-dessous sont un exemple : chez vous, `RejetLigne` peut être définie ailleurs que dans `erreurs.py` ; `grep` vous le dit.

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
ls solen/
grep -n "def " solen/*.py
grep -rn "class RejetLigne" solen/
grep -rn "quantite_invalide" solen/
```

**Exemple de code produit par Codex · à lire, pas à taper**

```
for num, ligne in lignes:                  # num = numéro dans le fichier
    lid = ligne["ligne_id"].strip()
    try:
        if not lid:
            raise RejetLigne("id_manquant", "ligne_id vide")
        if nb_id[lid] > 1:
            raise RejetLigne("id_conflit", "plusieurs lignes")
        if boutique not in boutiques or produit not in produits:
            raise RejetLigne("cle_inconnue", f"{boutique} / {produit}")
        q = texte_vers_quantite(ligne["quantite"])   # peut lever
    except RejetLigne as rej:
        rejets.ajouter("ventes.csv", num, lid, rej.motif)
```

L'ordre des `if` **est** l'ordre des règles. Chaque `raise` saute directement au bloc `except`, qui écrit la ligne dans le journal des rejets.

**Demande à Codex · texte à adapter**

```
Utilise $expliquer-bi. Ne modifie rien.
Où est la fonction qui rejette une ligne de ventes.csv ? Donne le fichier
et le numéro de ligne de chaque raise, puis l'endroit où l'exception est
rattrapée et écrite dans outputs/rejets.csv.
Suis la ligne 13 de l'échantillon (E-009-1, quantité -1).
Termine par une question pour vérifier que j'ai compris.
```

## Le jeu des trois questions (à deux)

B pose à A, qui répond en montrant l'écran : 1) où le CSV est-il lu ? 2) où un prix devient-il des millimes ? 3) que se passe-t-il si `ventes.csv` n'existe pas ? Puis on inverse.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2–M3 · Demander une modification précise

**Quoi :** faire modifier **une** fonction par Codex, en prévoyant l'effet avant d'envoyer. **Pourquoi :** lire du code ne suffit pas ; il faut savoir demander un changement précis, puis prouver qu'il fait ce qui était prévu, et rien d'autre. **Où :** même branche `lecture-nettoyage`, A au clavier, B prévoit avec lui.

**La modification :** un futur export de caisse pourrait écrire le code produit `p-01` (minuscules, tiret) au lieu de `P01` ; on veut l'accepter comme `P01`. **Prévoir, à deux :** combien de rejets du lot 1 changeront ? Regardez les données : `grep -ci ';p-[0-9]' tronc-commun/sources/lot1/ventes.csv` affiche `0`. Aucune ligne n'est écrite ainsi, et la règle casse/espaces transforme déjà `p01` en `P01`. Prévision : **aucun** rejet ne change (6 / 6 / 5 / 7, 1960 gardées) ; seul un test prouvera que `p-01` est accepté.

**Demande à Codex · texte à adapter**

```
Dans solen/, modifie seulement la fonction qui normalise les codes
produit : accepte aussi un code écrit p-01 (minuscules, tiret entre la
lettre et les chiffres) comme P01. Ne change aucune autre règle.
Ajoute un test pytest dans tests/test_codes.py : « p-01 », « p01 » et
« P01 » donnent tous P01. Lance pytest -q, puis python -m solen initial ;
recopie les sorties. Ma prévision : aucun rejet du lot 1 ne change
(6 / 6 / 5 / 7, 1960 gardées). Dis si elle est juste. Montre le diff.
Ne fais ni commit ni push.
```

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git diff
pytest -q
python -m solen initial
python -m controles.verifier --mission tronc-commun --etape rejets
```

## Contrôle et sortie attendue

Le diff ne touche que la fonction des codes produit et `tests/`. `pytest -q` : aucun `failed`. `solen initial` : toujours 1960 gardées, rejets 6 / 6 / 5 / 7 ; `--etape rejets` : `Résultat : 5 OK, 0 ÉCHEC`. Votre prévision est vérifiée. Lancez `pytest -q` à la racine du dépôt (`pytest tests` seul ne trouve pas le paquet `solen`). Le test part dans la même PR.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2-M4 · A prépare et ouvre la PR

**Quoi :** proposer la lecture et le nettoyage à la revue. **A** est auteur, **B** relecteur. **Pourquoi :** une deuxième paire d'yeux trouve ce que l'auteur ne voit plus, et chacun doit pouvoir expliquer le code fusionné.

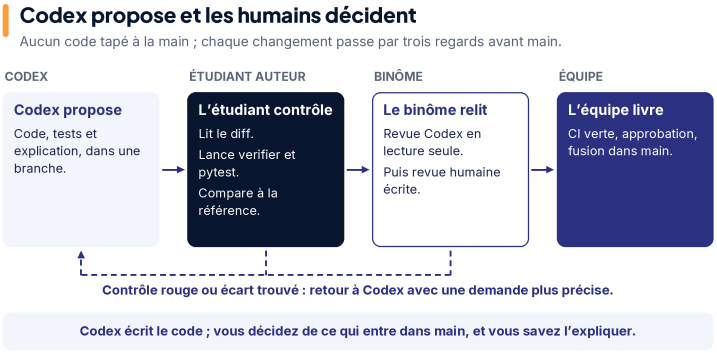



Schéma · Codex propose dans une branche ; l'auteur contrôle (diff, contrôles, référence) ; le binôme relit (Codex en lecture seule, puis revue humaine) ; l'équipe livre après CI verte et approbation.

**Demande à Codex · texte à adapter**

```
Utilise $preparer-pr pour la branche lecture-nettoyage.
Étape à contrôler : comptes de rejets de ventes.csv (lot1) contre
tronc-commun/reference/attendu_lot1.json ; l'entrepôt n'existe pas encore.
Relecteur : <prénom et compte GitHub de B>.
Ma phrase : « Cette PR permet de <…> », preuve : <commande>.
Fais la liste de contrôle, puis propose un titre et une description.
Ne commite pas, ne pousse pas, n'ouvre pas la PR.
```

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git status
git add solen/ tests/ notes/JOURNAL.md
git commit -m "Lire les sources et rejeter les lignes avec un motif"
git push -u origin lecture-nettoyage
```

Avant `git add` : `git status` ne doit montrer ni `outputs/` (ignoré par `.gitignore`), ni `.venv/`. Sur GitHub : **Compare & pull request**, collez la description, **Reviewers** = B, **Create pull request**.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2–M4 · B relit : Codex d'abord, puis vous

**Où :** poste de **B** : terminal, puis application Codex, puis GitHub.

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git fetch
git switch lecture-nettoyage
source .venv/bin/activate
python -m solen initial
python -m controles.verifier --mission tronc-commun --etape rejets
```

B **reproduit** la preuve sur son poste : mêmes comptes que dans la description de la PR. Puis une revue Codex en **lecture seule**, sous un angle précis :

**Demande à Codex · texte à adapter**

```
Utilise $revoir-bi. Branche : lecture-nettoyage. Mission : lis mission.txt.
Relis en lecture seule le diff de cette branche par rapport à main :
ne modifie rien, ne lance aucun chargement ni export.
Angle de la revue : données et rejets.
Rapporte au plus 3 défauts établis (fichier:ligne, règle violée,
exemple observable, correction proposée), puis les contrôles non réalisés.
J'ai moi-même un doute sur : <mon point>. Dis s'il est fondé, avec une preuve.
```

## Puis la revue humaine, sur GitHub

- **Files changed** : lisez au moins la fonction qui décide du motif et la conversion en millimes ; commentez une ligne précise.
- **Review changes** : **Approve** si tout est établi, **Request changes** si un défaut est **reproduit** (une valeur, une ligne, un compte).
- `git status` après la revue Codex : rien ne doit avoir changé.

### Une revue utile, exemple

« Ligne 42 de `solen/nettoyage.py` : une quantité `2,0` est acceptée alors que le cahier des charges dit « non entière → rejet ». Reproduit avec la ligne 1015 du lot 1 (`1.5`) : elle est bien rejetée, donc le cas `2,0` reste à vérifier. Question : faut-il un test ? »

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# S2–M4 · Corriger, relire, fusionner

Si B demande des changements, **A** corrige sur la **même branche** : un nouveau commit, un nouveau push, la PR se met à jour toute seule. Chaque constat reçoit une réponse : « corrigé dans <commit> » ou « non retenu parce que… ».

**Demande à Codex · texte à adapter**

```
Voici les constats de revue reçus sur ma PR <n> :
<coller les constats>
Pour chacun, vérifie d'abord contre le code actuel : établi, non établi,
ou non vérifiable, avec une preuve observable.
Corrige seulement les défauts établis, dans solen/, sans toucher
aux sources, aux références ni à controles/.
Relance python -m solen initial et recopie les comptes par motif.
Je prévois que <valeur> passera de <avant> à <après> : compare.
Ne fais ni commit ni push.
```

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git add <fichiers corrigés>
git commit -m "Corriger <le défaut> après revue"
git push
```

- **B** relit la correction (deuxième revue, éventuellement une nouvelle conversation Codex en lecture seule), puis **Approve**.
- CI verte (elle vérifie encore seulement l'installation) + approbation : **B** clique **Merge pull request**.
- Les deux : `git switch main` puis `git pull`.

## Erreurs courantes

- Corriger en modifiant `controles/` ou la référence : interdit ; le relecteur le verra dans le diff.
- Ouvrir une nouvelle PR pour la correction : inutile, la même branche suffit.

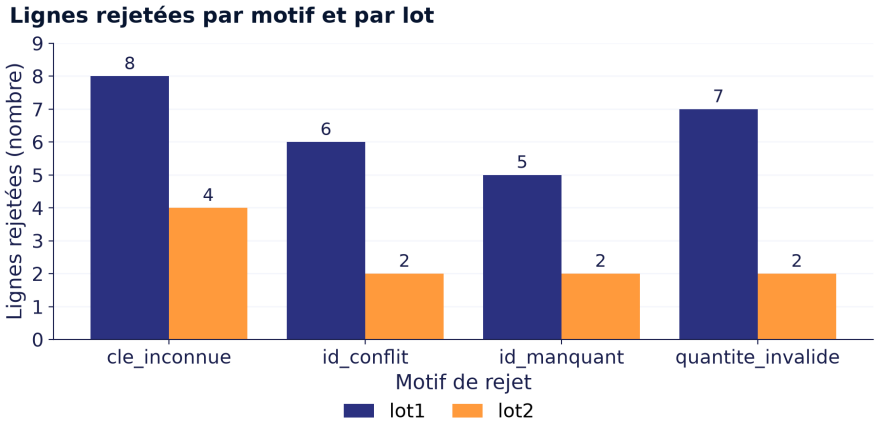
## L'entrée du journal de cette PR, exemple

**État actuel** : sources du lot 1 lues ; `ventes.csv` nettoyé. **Décision** : doublon exact ignoré avant tout autre test. **Preuve** : `python -m solen initial` puis `--etape rejets` → 5 OK, 0 ÉCHEC : 1960 gardées, rejets de `ventes.csv` 6 / 6 / 5 / 7 = `attendu_lot1.json` (le vérificateur ne contrôle encore que cette source). **Prochain objectif** : lire les retours en SQL et charger l'étoile ; preuve : `--etape initial` OK.

02 / TRONC COMMUN · SÉANCE 2 · LIRE ET NETTOYER

# Fin de séance 2 · Bilan

| Preuve | Où la voir |
|---|---|
| Lecture : 1998 lignes, 8 produits, 6 familles, 3 boutiques | `python -m solen sources` |
| Rejets `ventes.csv` lot 1 : 6 / 6 / 5 / 7 ; 14 doublons ; 1960 gardées | `outputs/rejets.csv` + `--etape rejets` |
| Ventes brutes des lignes gardées : 78 067,830 TND | sortie de `solen initial` |
| PR de A relue par B (Codex + humain), corrigée, fusionnée | GitHub |
| Entrée du journal avec preuve réelle | `notes/JOURNAL.md` |




Visuel de référence (`reference/visuels/`) : lignes rejetées par motif et par lot. Pour l'instant, seul le lot 1 vous concerne ; les 8 `cle_inconnue` incluent les 2 retours de la séance 3.

## À ce stade, je sais…

[ ] expliquer pourquoi le total de Karim (78 839,780 TND) est faux ;
[ ] citer les quatre motifs de rejet et l'ordre des règles ;
[ ] montrer, dans le code, la fonction qui rejette une ligne et l'exception utilisée ;
[ ] reproduire la preuve d'une PR avant de l'approuver.

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# Séance 3 · Les retours, puis l'étoile

Jeudi. Les ventes sont propres ; il manque les retours, donc les ventes nettes et la marge. Karim Jaziri vous montre l'outil de caisse.

### Épisode 4 · Ce que dit Karim

« Les retours ne sont pas dans un fichier : ils sont dans la base de la caisse, `operations.sqlite`. Certains ont été **annulés** par le magasin. Deux pointent vers des tickets que personne ne retrouve : `TK1999-1` et `TK2817-2`. Ne les inventez pas, ne les jetez pas en silence. Et rangez tout dans un vrai entrepôt : je veux pouvoir couper les chiffres par boutique, par zone et par mois sans tout recalculer. »

## Minutage de la séance 3 (240 min, estimations)

| Min | Format | Contenu |
|---|---|---|
| 10 | Explication | Épisode 4 : base opérationnelle contre entrepôt |
| 14 + 25 | Expl. + pratique | S3-M1 · SQL paramétré sur la base des retours |
| 16 + 15 | Expl. + pratique | S3-M2 · Grain, étoile, clés métier et techniques |
| 10 | Pause | — |
| 10 + 15 | Expl. + pratique | S3-M3 · La classe Entrepot |
| 6 + 35 | Expl. + pratique | S3-M4 · Chargement initial et vérificateur |
| 20 | Pratique | S3-M5 · Les totaux égaux à la référence |
| 10 | Pause | — |
| 5 + 34 | Expl. + pratique | S3-M6 · PR : B auteur, A relecteur |
| 5 + 10 | Expl. + pratique | S3-M7 · Choisir votre mission du catalogue |

Total : **66 min** d'explications, **154 min** de pratique, deux pauses. Preuve en fin de séance : `--etape initial` affiche **0 ÉCHEC** ; l'entrepôt a 5 tables reliées ; la mission du catalogue est choisie ; la PR de B est fusionnée.

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M1 · SQL : interroger la base des retours

**Quoi :** lire les retours dans la base SQLite **opérationnelle** (celle de la caisse) avec des requêtes SQL. **Pourquoi :** beaucoup de données d'entreprise vivent dans des bases, pas dans des fichiers. **Base opérationnelle** = sert à travailler au jour le jour (encaisser, rembourser) ; **entrepôt** = sert à analyser. On lit la première, on remplit le second, on ne modifie jamais la première.

| Mot SQL | Rôle | Exemple sur la table `retour` |
|---|---|---|
| `SELECT` | quelles colonnes | `SELECT numero_retour, montant_rembourse` |
| `FROM` | quelle table | `FROM retour` |
| `WHERE` | quelles lignes | `WHERE statut = 'VALIDE'` |
| `GROUP BY` | regrouper pour compter ou additionner | `SELECT statut, COUNT(*) … GROUP BY statut` |
| `ORDER BY` | trier | `ORDER BY id` |

**Action :** regarder la base **en lecture seule** (`-readonly`) avec `sqlite3`, installé en séance 0. `DB=…` crée une **variable du terminal** : ensuite, `$DB` est remplacé par ce chemin. Ces deux requêtes sont des commandes de contrôle fournies : copiez-les, ne les modifiez pas.

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
DB=tronc-commun/sources/lot1/operations.sqlite
sqlite3 -readonly $DB ".tables"
sqlite3 -readonly $DB "SELECT statut, COUNT(*) FROM retour GROUP BY statut;"
```

**Sortie attendue · ne pas recopier**

```
motif_retour  parametre     retour
ANNULE|4
VALIDE|78
```

82 retours : 78 VALIDE, 4 ANNULE. Prévoyez : combien de retours VALIDE en mars 2026 (aucun, une dizaine, une centaine) ? Notez votre estimation.

## Erreurs courantes

- Comparer des dates de formats différents : vérifiez que `date_retour` est bien écrit `AAAA-MM-JJ hh:mm:ss`.
- Après chargement, mars ne vaudra que **875,310 TND** : le retour `R26-00039` (18,500 TND) vise `TK1999-1`, inconnu → rejet `cle_inconnue`. Ce n'est pas une erreur.

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M1 · SQL paramétré depuis Python

**Où :** poste de **B**, qui est auteur à cette séance ; A à côté. Terminal, puis Codex.

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git switch main
git pull
git switch -c etoile-initial
```

**Demande à Codex · texte à adapter**

```
Dans solen/, ajoute une fonction qui lit les retours de la base
tronc-commun/sources/lot1/operations.sqlite avec le module sqlite3, en
lecture seule. La requête SQL est paramétrée avec ? (jamais de f-string
ni de concaténation pour les valeurs) et accepte un statut, une date de
début (incluse) et une date de fin (exclue).
Lance-la pour les retours VALIDE de 2026-03 et affiche : nombre de lignes,
somme de montant_rembourse en millimes. Montre la requête et explique
SELECT, FROM, WHERE, ORDER BY. Explique en 3 lignes pourquoi ? protège
mieux qu'une chaîne construite. Ne fais ni commit ni push.
```

**Sortie attendue · ne pas recopier**

```
retours VALIDE 2026-03 : 25 lignes · 893810 millimes (893,810 TND)
```

**Exemple de code produit par Codex · à lire, pas à taper**

```
def requete_sqlite(chemin, sql, parametres=()):
    """Base ouverte en lecture seule ; requête toujours paramétrée."""
    con = sqlite3.connect(f"file:{chemin}?mode=ro", uri=True)
    con.row_factory = sqlite3.Row
    try:
        return [dict(r) for r in con.execute(sql, parametres)]
    finally:
        con.close()

sql = "SELECT * FROM retour WHERE statut = ? AND date_retour >= ?" \
      " AND date_retour < ? ORDER BY id"
mars = requete_sqlite(base, sql, ("VALIDE", "2026-03-01", "2026-04-01"))
```

Le ? est une **case** : la valeur est envoyée à part et ne peut jamais être lue comme du SQL. Une chaîne construite à la main avec une valeur comme `NBE' OR '1'='1` changerait le sens de la requête. **Borne de fin exclue** (`< 2026-04-01`) : un retour du 31 mars à 18 h est compté une fois, en mars.

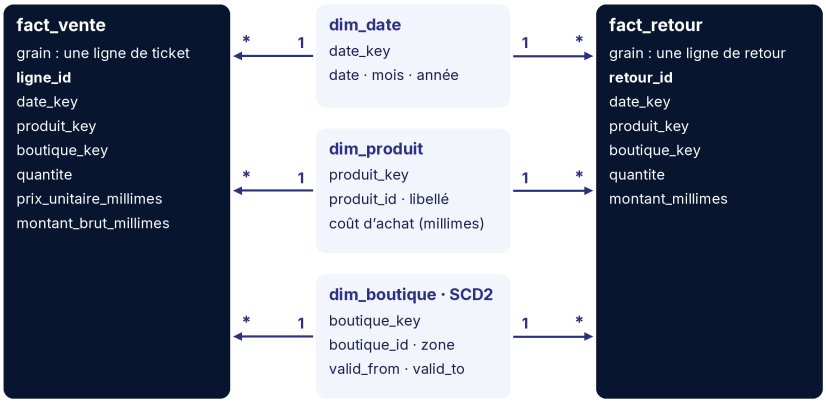
02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M2 · Grain et schéma en étoile

**Quoi :** organiser l'entrepôt en **étoile** : des tables de **faits** au centre, des **dimensions** autour. **Pourquoi :** la direction veut couper les mêmes chiffres par boutique, par zone, par mois, par famille. Chaque coupe est une dimension ; les montants à additionner sont dans les faits.

**L'étoile de la mission 00**

Trois dimensions partagées, deux tables de faits ; chaque relation va de 1 vers plusieurs.




**1 → * : une ligne de dimension décrit plusieurs lignes de fait ; la flèche suit le sens du filtre.**

Chaque flèche relie la clé de même nom : date_key, produit_key, boutique_key.

Les dimensions sont chargées avant les faits ; les montants sont des millimes entiers.

Schéma · L'étoile de la mission 00 : `fact_vente` et `fact_retour` partagent trois dimensions. Chaque flèche va d'une dimension (1) vers les faits (plusieurs) : une boutique décrit des centaines de ventes.

| Table | Rôle | Une ligne = | Lot 1 |
|---|---|---|---|
| `dim_date` | dimension | un jour | 181 (1er janvier → 30 juin) |
| `dim_produit` | dimension | un produit du catalogue | 8 |
| `dim_boutique` | dimension **SCD2** | une **version** de boutique | 3 |
| `fact_vente` | fait | une ligne de ticket acceptée (le grain) | 1960 |
| `fact_retour` | fait | un retour VALIDE accepté | 76 |

**Fait** = un événement qui se mesure (quantité, montant). **Dimension** = ce qui le décrit : quand, quoi, où. Exemple : la vente `TK10001-2` (fait) a lieu le 2026-01-01 (dim_date), porte sur P01 (dim_produit), à TUN (dim_boutique).

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M2 · Clé métier, clé technique

Une **clé métier** vient des sources et a un sens pour l'entreprise (`NBE`, `P01`). Une **clé technique** est un entier attribué par l'entrepôt. On en a besoin dès qu'une même boutique a **plusieurs versions** : NBE en aura deux au lot 2.




Schéma · Une boutique, deux versions, donc deux clés techniques ; les faits pointent vers `boutique_key`. Schéma d'explication : les numéros 2 et 5 et la date 2026-01-01 sont des exemples. Dans vos données, NBE est en Cap Bon depuis le 2021-05-10 et les numéros sont ceux que votre code attribue.

| Objet | Clé métier (texte) | Clé technique (entier) |
|---|---|---|
| Produit | `produit_id` = `P01` | `produit_key` = 1, 2, 3… |
| Boutique | `boutique_id` = `NBE` | `boutique_key` : une par **version** |
| Jour | `date_complete` = `2026-03-04` | `date_key` = 20260304 |
| Vente | `ligne_id` = `TK10001-2` (clé unique du fait) | — |
| Retour | `retour_id` = `R26-00001` | — |

**Dimensions avant faits** : une vente ne peut pointer que vers une `boutique_key` qui existe déjà. Les `FOREIGN KEY` (clés étrangères) déclarées dans SQLite le vérifient : le vérificateur affichera `PRAGMA foreign_key_check : 0 violation`.

## Erreurs courantes

- Relier les faits par `boutique_id` (texte) : dès que NBE aura deux versions, ses ventes seront comptées deux fois.

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M3 · La classe Entrepot

**Quoi :** comprendre pourquoi Codex range l'accès à l'entrepôt dans une **classe**. **Pourquoi :** c'est le seul endroit où la programmation objet sert vraiment dans ce projet : une seule porte d'entrée vers la base, et un chargement **tout ou rien**.

| Mot | En une phrase | Ici |
|---|---|---|
| **Classe** | Un plan qui décrit des données et des actions. | `class Entrepot` |
| **Objet** | Une chose construite à partir du plan. | e, l'entrepôt ouvert sur `outputs/entrepot.sqlite` |
| **Méthode** | Une action de l'objet, appelée avec un point. | `e.inserer("fact_vente", …)`, `e.compter("fact_vente")` |
| **Transaction** | Un lot d'écritures validé en entier ou annulé en entier. | `commit` si tout va bien, `rollback` sinon |

**Exemple de code produit par Codex · à lire, pas à taper**

```
class Entrepot:
    """Accès à outputs/entrepot.sqlite : with Entrepot(chemin) as e:"""
    def __enter__(self):
        self.con = sqlite3.connect(self.chemin)
        self.con.execute("PRAGMA foreign_keys = ON")
        return self

    def __exit__(self, typ, val, tb):
        if typ is None:
            self.con.commit()      # tout s'est bien passé : on valide
        else:
            self.con.rollback()    # une erreur : on annule tout
```

`with Entrepot(chemin) as e:` ouvre la base ; à la fin du bloc, `__exit__` valide ou annule. Si une exception arrive au milieu des faits, **rien** n'est écrit : pas d'entrepôt à moitié chargé.

**Demande à Codex · texte à adapter**

```
Utilise $expliquer-bi. Ne modifie rien.
Explique la classe Entrepot de notre code : où elle ouvre la base, où
elle valide, où elle annule. Que contient fact_vente si une erreur
survient pendant le chargement des retours ? Laisse-moi prévoir
avant de donner la réponse.
```

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M4 · Chargement initial : la demande

**Quoi :** `python -m solen initial` recrée l'entrepôt depuis zéro avec le lot 1. **Pourquoi :** c'est la première version complète des chiffres, et le premier contrôle complet du vérificateur. **Prévoir** (sur papier, avant d'envoyer) :

| Table | Votre prévision | Pourquoi |
|---|---|---|
| `dim_date` | … | un jour par ligne du 2026-01-01 au 2026-06-30 |
| `dim_boutique` | … | trois boutiques, une version chacune au lot 1 |
| `fact_vente` | … | lignes gardées en séance 2 |
| `fact_retour` | … | 78 VALIDE − 2 vers une ligne inconnue |
| rejets `cle_inconnue` | … | 6 ventes + 2 retours |

**Demande à Codex · texte à adapter**

```
Lis tronc-commun/cahier_des_charges.md (§ 4 à 6) et
tronc-commun/reference/contrat_entrepot.json. Dans solen/, crée une classe
Entrepot qui gère outputs/entrepot.sqlite, et complète python -m solen
initial : recréer l'entrepôt depuis zéro avec lot1 ; d'abord les dimensions
(dim_date, dim_produit, dim_boutique), puis les faits (fact_vente, puis
fact_retour avec les retours VALIDE) ; clés *_key entières, clés *_id
texte, montants en millimes entiers, FOREIGN KEY déclarées, transaction
tout ou rien. dim_boutique a valid_from et valid_to (NULL = courante).
Retour vers une ligne inconnue : rejets.csv, cle_inconnue,
numero_ligne = retour.id. Lance python -m solen initial puis
python -m controles.verifier --mission tronc-commun --etape initial
et recopie les sorties. J'attends <mes prévisions> : compare.
Explique le grain de chaque table de faits. Ne fais ni commit ni push.
```

## Si Codex propose d'abord un plan

C'est normal : `AGENTS.md` lui demande un plan avant un changement de plus de ~50 lignes. Choisissez **une** étape (par exemple les dimensions), contrôlez, puis la suivante. Ne répondez pas « fais tout ».

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M4 · Contrôle : --etape initial

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
python -m solen initial
python -m controles.verifier --mission tronc-commun --etape initial
sqlite3 -readonly outputs/entrepot.sqlite "SELECT COUNT(*) FROM fact_vente;"
```

**Sortie attendue · ne pas recopier · extrait**

```
== Étape initial : comparaison avec attendu_lot1.json ==
OK — fact_vente : 1960 lignes
OK — fact_retour : 76 lignes
OK — PRAGMA foreign_key_check : 0 violation (6 clés étrangères déclarées)
OK — dim_boutique : NBE le 2026-07-15 → zone « Cap Bon »
OK — rejets lot1 : cle_inconnue = 8
OK — rejets lot1 · ventes.csv : 24 numero_ligne réels (2 à 1999), uniques
OK — Ventes nettes : 74309500 millimes (74 309,500 TND)
OK — Taux de retour : 0,048142 (4,81 %)
OK — agrégats par_zone : 3 valeurs de « zone » × 7 indicateurs conformes

Résultat : 51 OK, 0 ÉCHEC.
1960
```

Remarquez : au lot 1, NBE est **encore** en Cap Bon le 2026-07-15. Normal : le changement de zone n'arrive qu'avec le lot 2. Un ÉCHEC dit toujours **attendu** puis **observé** ; il se lit comme un indice :

| Message (extrait réel du format) | Cause probable |
|---|---|
| `ÉCHEC — fact_vente : attendu 1960 lignes, observé 1974` | 1974 = 1960 + 14 : les doublons exacts sont gardés. |
| `ÉCHEC — fact_vente : attendu des entiers et aucune valeur vide, observé montant_brut_millimes (… valeurs non entières)` | montants stockés en décimaux, pas en millimes entiers. |
| `ÉCHEC — rejets lot1 : cle_inconnue, attendu 8, observé 6` | les 2 retours vers un ticket inconnu ne sont pas écrits dans `rejets.csv`. |

Un échec ? Lisez la **première** ligne ÉCHEC, puis utilisez la fiche `prompts/10-erreur.md` : message exact collé, hypothèse, diagnostic **avant** correction.

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M5 · Les totaux égaux à la référence

**Quoi :** vérifier que vous savez **refaire** les chiffres, pas seulement lire OK. **Pourquoi :** devant la direction, « le vérificateur est vert » ne suffit pas ; il faut savoir d'où vient chaque nombre. **Où :** papier, calculatrice, et `tronc-commun/reference/README.md`.

| Indicateur, lot 1 | Référence | À refaire à la main |
|---|---|---|
| Ventes brutes | 78 067,830 TND | — |
| Retours | 3 758,330 TND | — |
| Ventes nettes | **74 309,500 TND** | 78 067,830 − 3 758,330 |
| Coût d'achat des unités nettes | 45 317,800 TND | — |
| Marge sur coût d'achat | **28 991,700 TND** | 74 309,500 − 45 317,800 |
| Taux de retour | **4,81 %** | 3 758 330 / 78 067 830 = 0,0481 |

| Zone (boutique) | Ventes nettes | Marge | Taux de retour |
|---|---|---|---|
| Grand Tunis (TUN) | 34 686,420 | 13 421,220 | 2,48 % |
| Cap Bon (NBE) | 19 262,780 | 7 577,880 | 4,14 % |
| Sud (SFX) | 20 360,300 | 7 992,600 | **9,12 %** |
| Total | 74 309,500 | 28 991,700 | 4,81 % |

**Action :** additionnez les trois zones : vous devez retrouver 74 309,500. Puis repérez deux faits surprenants : Sfax rend presque 4 fois plus que Tunis, et juin atteint **15,88 %** de retours (1 435,700 TND sur 9 042,420).

**Demande à Codex · texte à adapter**

```
Utilise $expliquer-bi. Ne modifie rien.
Le taux de retour de juin 2026 vaut 15,88 % dans la référence. À partir de
outputs/entrepot.sqlite (lecture seule), donne les retours de juin par
boutique et par famille, en TND, et le fichier:ligne qui produit ce calcul.
Ma prévision : <boutique et famille>. Dis si j'ai raison.
```

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M6 · PR : B auteur, A relecteur

Même cycle qu'en séance 2, **rôles inversés**. B a écrit les demandes ; A doit pouvoir expliquer le code fusionné aussi bien que B.

| Qui | Geste | Preuve |
|---|---|---|
| B | `$preparer-pr` (étape `initial`), puis `git add solen/ notes/JOURNAL.md`, `git commit`, `git push -u origin etoile-initial`, PR, Reviewers = A | description avec `Résultat : 51 OK, 0 ÉCHEC.` |
| A | `git fetch`, `git switch etoile-initial`, `python -m solen initial`, vérificateur | le même `51 OK` sur **son** poste |
| A | `$revoir-bi`, angle **modèle et clés** | ≤ 3 défauts établis, contrôles non réalisés séparés |
| A | revue GitHub : Approve ou Request changes | un commentaire sur une ligne |
| B | correction sur la même branche si besoin | nouveau commit, contrôles relancés |
| A | **Merge pull request** ; puis les deux `git switch main`, `git pull` | PR fusionnée |

## Ce que A vérifie lui-même dans Files changed

- Les dimensions sont chargées **avant** les faits (ordre des appels).
- Chaque requête SQL qui reçoit une valeur utilise `?` ; aucune f-string avec une valeur dans un `execute`.
- Les montants sont des entiers (`int`), jamais des `float` stockés.
- Rien n'est modifié dans `tronc-commun/`, `reference/` ou `controles/`.

### Et la CI ?

Elle reste à `ETAPE_CI: aucune` jusqu'à la séance 4 : elle ne vérifie que l'installation. Vous l'avancerez quand le pipeline saura aussi charger le lot 2 (S4-M3).

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# S3-M7 · Choisir votre mission du catalogue

**Quoi :** choisir **une** mission parmi dix, pour le dernier bloc de la séance 4, la séance 5 et le travail personnel. **Pourquoi :** vous referez seuls, sur un autre métier, tout ce que le tronc commun vous a appris. Le livret `04_catalogue_missions.pdf` décrit chacune ; lisez au moins deux fiches avant de décider.

| # | Entreprise fictive | Grain du fait | Historique (SCD2) |
|---|---|---|---|
| M01 | Carthage Market, magasins | ligne de ticket | magasin → région |
| M02 | Souk Express, e-commerce | ligne de commande | canal → famille |
| M03 | Medina Distribution, B2B | ligne de facture | client → segment |
| M04 | Colis Rapide, livraison | colis livré | agence → zone |
| M05 | Académie Nour, formation | présence | session → centre |
| M06 | Dar Azur, hôtellerie | séjour (nuitées) | canal → commission |
| M07 | FibreNet, abonnements | facture mensuelle | offre → gamme / prix |
| M08 | TechniServ, maintenance | intervention | équipement → contrat |
| M09 | Scène Libre, événementiel | billet vendu | canal → frais |
| M10 | Pixel Studio, agence | ligne de temps | projet → chef de projet |

- Choisissez un métier que vous savez **expliquer** avec un exemple de la vie réelle.
- Notez le choix dans `notes/JOURNAL.md` (rubrique **Décision**) et dites-le au professeur.
- **Ne changez pas encore** `mission.txt` : il reste `tronc-commun` jusqu'au dernier bloc de la séance 4 (« Passer à votre mission »).

## Ce qui changera, ce qui ne changera pas

Chaque mission du catalogue a la même forme que le tronc commun : `fiche_mission.md`, `cahier_des_charges.md`, `dictionnaire.md`, un `echantillon/` avec `attendu.md`, deux lots, une base SQLite opérationnelle, une dimension historisée et un dossier `reference/`. Les mots changent (colis, nuitées, abonnés…), la méthode reste : échantillon à la main, petites étapes, contrôles, PR relues.

02 / TRONC COMMUN · SÉANCE 3 · BASE OPÉRATIONNELLE ET ÉTOILE

# Fin de séance 3 · Bilan

| Preuve | Où la voir |
|---|---|
| Retours VALIDE de mars : 25 lignes, 893,810 TND en source | fonction SQL paramétrée |
| `--etape initial` : 51 OK, 0 ÉCHEC | terminal de A **et** de B |
| dim_date 181 · dim_produit 8 · dim_boutique 3 · fact_vente 1960 · fact_retour 76 | vérificateur, `sqlite3 -readonly` |
| Ventes nettes 74 309,500 · marge 28 991,700 · taux 4,81 % | vérificateur + calcul à la main |
| PR de B relue par A, fusionnée ; mission du catalogue choisie | GitHub, journal |

## À ce stade, je sais…

[ ] expliquer la différence entre base opérationnelle et entrepôt ;
[ ] lire une requête SQL et dire pourquoi elle utilise ? ;
[ ] dessiner l'étoile de mémoire, avec le grain de chaque fait ;
[ ] distinguer clé métier et clé technique, avec NBE comme exemple ;
[ ] dire ce que fait `with Entrepot(...)` quand une erreur arrive ;
[ ] refaire à la main ventes nettes, marge et taux de retour du lot 1.

## Avant la séance 4 (≈ 1 h, à deux : compté dans les 4 h de la mission)

Lisez la fiche et le guide de votre mission, puis calculez son **échantillon à la main** avant de lire la correction. Ouvrez aussi `tronc-commun/sources/lot2/boutiques.json` : trouvez la ligne qui annonce Nord-Est, et prévoyez combien de lignes aura `dim_boutique` après le lot 2.

Gardez une note d'état de fin de séance (fiche `prompts/16-reprendre.md`), comme en séance 1.

## À préparer à deux

Suivez à voix haute une ligne de `ventes.csv` jusqu'à l'entrepôt : `TK10001-2` (gardée), puis `TK10111-9` (rejetée). Pour chacune : quelle règle, quelle table, quelle clé, quel montant en millimes.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# Séance 4 · Juillet arrive, Nabeul change

### Épisode 5 · Mail de Lamia Ben Salah

« Voici l'export de juillet et août (`lot2`). Rappel : Nabeul est en zone Nord-Est depuis le 15 juillet. Le service caisse me dit que l'export a recopié les ventes des 29 et 30 juin et quelques retours de fin juin. Je ne veux **aucun doublon**, et les chiffres du premier semestre ne doivent **pas bouger**. »

Karim ajoute : « Et avant de livrer quoi que ce soit, je veux voir que vous savez trouver un défaut que vous n'avez pas écrit, le corriger, et le prouver. » (Épisode 6, la revue.)

## Minutage de la séance 4 (240 min, estimations)

| Min | Format | Contenu |
|---|---|---|
| 10 + 50 | Expl. + pratique | Épisode 5 · S4-M1 · Incrément et historique SCD2 |
| 10 + 35 | Expl. + pratique | S4-M2 · Rejeu et journal des chargements · S4-M3 · Tests, CI |
| 10 | Pause | — |
| 10 + 30 | Expl. + pratique | S4-M4 · Épisode 6 : revue → correction → revue |
| 10 + 15 | Démo + pratique | S4-M5 · Conflit : démonstration du professeur, puis reproduction |
| 10 | Pause | — |
| 10 + 40 | Expl. + pratique | Passer à votre mission : S4-M7 export et livraison du tronc commun ; S4-M8 changement de mission, première PR |
| — | Facultatif | S4-M6 · Un Skill local (extension, ou travail personnel) |

Total : **50 min** d'explications (dont 10 min de démonstration du conflit), **170 min** de pratique, deux pauses. Preuve en fin de séance : lot 2 chargé, historique préservé, rejeu prouvé, CI verte, un défaut trouvé et corrigé, un conflit résolu, la livraison du tronc commun commitée dans `livraison/tronc-commun/`, la première PR de votre mission ouverte.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4-M1 · Garder l'historique : SCD2

**Quoi :** quand NBE change de zone, **ajouter** une version au lieu d'écraser l'ancienne. **Pourquoi :** la direction veut que juin reste dans Cap Bon. Si l'on remplace « Cap Bon » par « Nord-Est », tout l'historique de Nabeul bascule et les chiffres du premier semestre changent. **SCD2** (dimension à variation lente, type 2) = une ligne par version, avec ses dates de validité.

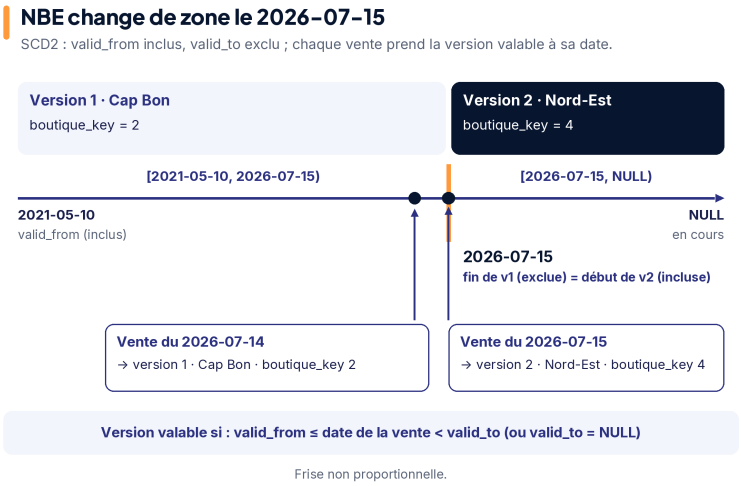



Schéma · NBE change de zone le 2026-07-15 : la vente du 14 juillet garde la version Cap Bon, celle du 15 prend la version Nord-Est. Schéma d'explication : clés 2 et 5 et date de début d'exemple ; dans vos données, la version Cap Bon commence le 2021-05-10.

| boutique_id | zone | valid_from (inclus) | valid_to (exclu) |
|---|---|---|---|
| NBE | Cap Bon | 2021-05-10 | 2026-07-15 |
| NBE | Nord-Est | 2026-07-15 | (vide = courante) |
| SFX | Sud | 2022-09-19 | (vide) |
| TUN | Grand Tunis | 2018-04-02 | (vide) |

Une version est valable un jour donné si `valid_from <= jour` **et** (`valid_to` vide **ou** `jour < valid_to`). D'où les trois dates frontières que le vérificateur contrôle : NBE le **2026-06-30** → Cap Bon, le **2026-07-14** → Cap Bon, le **2026-07-15** → Nord-Est.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4-M1 · La demande d'incrément

**Où :** poste de **A** (auteur), branche neuve. **Prévoir** sur papier : (1) lignes de `dim_boutique` après le lot 2 ; (2) zone d'une vente NBE du 14 puis du 15 juillet ; (3) lignes de `fact_vente` après **deux** `increment`.

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git switch main
git pull
git switch -c increment-scd2
```

**Demande à Codex · texte à adapter**

```
Ajoute la commande python -m solen increment qui charge
tronc-commun/sources/lot2/ dans l'entrepôt existant, sans effacer lot1.
Règles (cahier des charges § 7 et § 8) :
- SCD2 sur dim_boutique : quand la zone change, fermer l'ancienne version
  (valid_to = date du changement, exclue) et créer une nouvelle version
  (valid_from = même date, incluse, valid_to NULL) ;
- un fait est rattaché à la version valable à SA date ; ne jamais
  modifier les faits anciens ; dim_date prolongée jusqu'au 2026-08-31 ;
- rejeu : une vente ou un retour dont la clé métier est déjà dans
  l'entrepôt est ignoré ; relancer increment n'ajoute aucune ligne ;
- les rejets de lot2 remplacent ceux de lot2 et gardent ceux de lot1.
Lance initial, increment, le vérificateur --etape increment, puis
increment encore et --etape rejeu. Recopie les sorties.
Montre la requête qui choisit la version de boutique pour une date.
Mes prévisions : <1>, <2>, <3>. Compare. Ne fais ni commit ni push.
```

**Exemple de code produit par Codex · à lire, pas à taper**

```
SELECT boutique_key FROM dim_boutique
WHERE boutique_id = ? AND valid_from <= ?
  AND (valid_to IS NULL OR valid_to > ?)
```

Lisez la requête à voix haute avec la date 2026-07-15 : la version Cap Bon a `valid_to` = 2026-07-15, qui n'est **pas** > 2026-07-15 ; elle est exclue. Seule Nord-Est reste. Fermer l'ancienne version demande un `UPDATE` de `valid_to` : c'est normal. Ce qui est interdit, c'est un `UPDATE` de la **zone** (défaut 2 de S4-M4).

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M2 · Rejeu : relancer sans rien ajouter

**Quoi :** relancer le même lot 2 ne doit rien changer. **Pourquoi :** en vrai, un chargement se relance souvent (panne, export refait). Une **insertion idempotente** (relancée, elle donne le même résultat) repose sur la **clé métier** : `ligne_id` déjà présent → ligne ignorée.

| Lot 2 | Lues | Ignorées | Rejetées | Ajoutées | Après |
|---|---|---|---|---|---|
| `ventes.csv` | 413 | 4 doublons + 15 déjà chargées (29-30 juin) | 9 | 385 | fact_vente **2345** |
| `operations.sqlite` | 29 | 2 ANNULE + 3 déjà chargés | 1 | 23 | fact_retour **99** |
| Dimensions | — | — | — | 62 jours, 1 version | dim_date 243, dim_boutique 4 |

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
python -m solen initial
python -m solen increment
python -m controles.verifier --mission tronc-commun --etape increment
python -m solen increment
python -m controles.verifier --mission tronc-commun --etape rejeu
grep -c "lot2" outputs/rejets.csv
```

**Sortie attendue · ne pas recopier · extrait**

```
INFO — empreinte enregistrée : outputs/verification/empreinte_lot2.json
Résultat : 51 OK, 0 ÉCHEC.
OK — dim_boutique : NBE le 2026-07-15 → zone « Nord-Est »
OK — rejeu : 5 tables identiques à l'empreinte de --etape increment
Résultat : 52 OK, 0 ÉCHEC.
10
```

L'**empreinte** (lignes et somme de contrôle de chaque table), enregistrée par `--etape increment`, est comparée par `--etape rejeu` : lancez `--etape increment` **avant** de relancer, sinon `ÉCHEC — rejeu : empreinte absente`. Rejets du lot 2 : toujours 10 (relancer un lot remplace ses rejets).

## Erreurs courantes

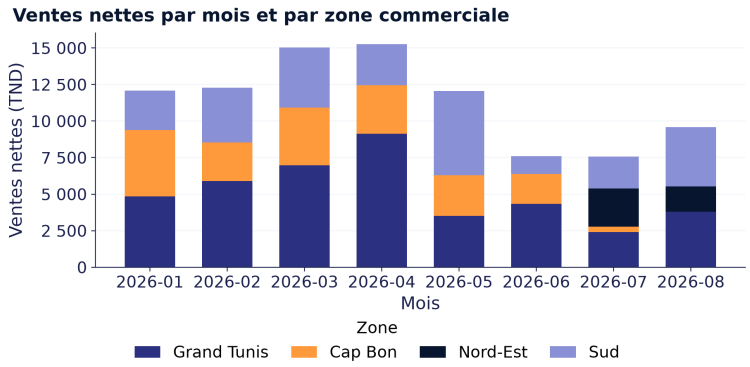
• `UPDATE … SET zone` sur la ligne existante : le vérificateur répond `dim_boutique : version attendue absente ('NBE', 'Cap Bon', '2021-05-10', '2026-07-15')`.
• `fact_vente : attendu 2345 lignes, observé 2360` : les 15 ventes recopiées des 29-30 juin sont chargées deux fois ; la clé métier n'est pas testée.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4-M2 · Les chiffres après le lot 2

| Indicateur | Après lot 1 | Après lot 2 (= après rejeu) |
|---|---|---|
| Ventes brutes | 78 067,830 TND | 95 849,090 TND |
| Retours | 3 758,330 TND | 4 404,630 TND |
| Ventes nettes | 74 309,500 TND | **91 444,460 TND** |
| Marge sur coût d'achat | 28 991,700 TND | **35 606,860 TND** |
| Taux de retour | 4,81 % | **4,60 %** |
| Unités nettes | 3605 | 4264 |

| Zone | Ventes nettes | Taux de retour |
|---|---|---|
| Grand Tunis | 40 887,210 | 3,11 % |
| Cap Bon | 19 653,980 | 4,06 % |
| Nord-Est | 4 323,400 | 3,01 % |
| Sud | 26 579,870 | 7,40 % |




Visuel de référence : ventes nettes par mois et par zone (TND). Nord-Est n'apparaît qu'à partir de juillet ; janvier à juin sont identiques à l'état après lot 1.

## Exercice à deux

Cap Bon passe de 19 262,780 à 19 653,980 TND alors que Nabeul est en Nord-Est depuis le 15 juillet. D'où viennent ces 391,200 TND ? (Indice : du 1er au 14 juillet.) Vérifiez : 19 653,980 + 4 323,400 = 23 977,380 = ventes nettes de NBE.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M2 · Garder un journal des chargements

**Quoi :** garder une trace de **chaque** chargement dans `outputs/journal_chargements.json`. **Pourquoi :** l'écran « Qualité des données » de l'application (séance 5) affiche une ligne par chargement : lot 1, lot 2, puis le rejeu du lot 2 à 0 ajout. Le pipeline affiche ces comptes puis les oublie ; sans journal, l'application n'a rien à lire. **Où :** branche `increment-scd2`, poste de A.

**Demande à Codex · texte à adapter**

```
Fais écrire par python -m solen initial et increment un journal des
chargements : outputs/journal_chargements.json, une liste avec une entrée
par chargement : lot, date et heure, et pour chaque source : lues,
doublons ignorés, déjà chargées, ignorées (ANNULE), rejetées, ajoutées.
initial recrée le fichier ; increment ajoute une entrée, même s'il
n'ajoute aucune ligne. Ne change aucun compte ni aucune table.
Montre le fichier après initial, increment, increment.
Ne fais ni commit ni push.
```

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
python -m solen initial
python -m solen increment
python -m controles.verifier --mission tronc-commun --etape increment
python -m solen increment
python -m controles.verifier --mission tronc-commun --etape rejeu
python -m json.tool outputs/journal_chargements.json | head -40
```

| Entrée du journal | ventes.csv : ajoutées | operations.sqlite : ajoutés |
|---|---|---|
| 1 · lot1 (`initial`) | 1960 | 76 |
| 2 · lot2 (`increment`) | 385 | 23 |
| 3 · lot2 relancé (rejeu) | 0 | 0 |

## Contrôle

Trois entrées, dans l'ordre ; la troisième n'ajoute rien. Les contrôles ne bougent pas : `--etape increment` 51 OK, `--etape rejeu` 52 OK. Le journal vit dans `outputs/` : il n'est pas commité, et `initial` le recrée pour chaque mission.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M3 · Des tests écrits par Codex

**Quoi :** des tests automatiques sur **vos** règles, dans `tests/`. **Pourquoi :** le vérificateur compare des totaux ; un test dit **quelle règle** est cassée. Un **test pytest** est une petite fonction qui prépare un cas calculé à la main, appelle le code et vérifie le résultat avec `assert`. S'il échoue, `pytest` l'affiche en rouge.

**Exemple de code produit par Codex · à lire, pas à taper**

```
import pytest
from solen.conversions import texte_vers_millimes, texte_vers_quantite
from solen.erreurs import RejetLigne

def test_une_quantite_negative_est_rejetee():
    with pytest.raises(RejetLigne) as erreur:
        texte_vers_quantite("-2")
    assert erreur.value.motif == "quantite_invalide"

def test_un_prix_avec_espace_et_virgule_devient_des_millimes():
    assert texte_vers_millimes("1 240,000") == 1240000
```

Un bon test part d'un exemple **calculé à la main** (1 240,000 TND = 1 240 000 millimes), jamais du résultat du code (`assert total == total` ne peut pas échouer).

**Demande à Codex · texte à adapter**

```
Dans tests/ (pas dans controles/), écris 4 tests pytest courts :
une quantité -2 est rejetée (quantite_invalide) ; « 1 240,000 » vaut
1240000 millimes ; une vente NBE du 2026-07-14 est en Cap Bon et une du
2026-07-15 en Nord-Est ; relancer increment n'ajoute aucune ligne.
Noms en français qui disent la règle ; petits jeux de données construits
dans le test, pas les sources complètes. Lance pytest -q, recopie la
sortie. Puis, sans le garder, montre quel test échouerait si l'on
acceptait les quantités négatives, et pourquoi. Ne fais ni commit ni push.
```

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
pytest -q
```

**Sortie attendue · ne pas recopier**

```
..............ss
<n> passed, 2 skipped
```

Les 2 tests ignorés sont ceux de l'export et de l'application : pas encore construits. Lancez `pytest -q` à la racine du dépôt : `pytest tests` seul ne trouve pas le paquet `solen`.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4-M3 · La CI avance d'un cran

**CI** (intégration continue) : à chaque PR, GitHub prend une machine Linux neuve, installe le projet depuis zéro et lance les contrôles. Si c'est vert là-bas, le résultat ne dépend pas de « ça marche chez moi ». La variable `ETAPE_CI` de `.github/workflows/controles.yml` dit jusqu'où aller.

| ETAPE_CI | Quand | Ce que la CI ajoute |
|---|---|---|
| aucune | S1 → S3 | installation seulement |
| `rejets` | Séance 2 | `python -m solen initial` puis `verifier --etape rejets` : seuls les rejets du CSV de faits sont jugés. |
| `initial` / `increment` | si une étape résiste | `solen initial`, puis `solen increment`, et le vérificateur de l'étape |
| `rejeu` | **maintenant** (S4-M3) | relance `increment`, vérificateur `--etape rejeu` |
| `export` | S4-M7, si l'export résiste | `solen export`, vérificateur `--etape export` |
| `tout` | S4-M7 (export) | tout le parcours + `pytest -q` (vos tests compris) |

**Demande à Codex · texte à adapter**

```
Mets à jour .github/workflows/controles.yml : passe ETAPE_CI de aucune
à rejeu, et rien d'autre. Explique en 5 lignes ce que la CI exécutera
à la prochaine PR, étape par étape. Aucun secret, aucun service externe.
Ne fais ni commit ni push.
```

**Contrôle :** `git diff .github/workflows/controles.yml` montre une seule ligne changée : `ETAPE_CI: aucune` devient `ETAPE_CI: rejeu`. Après le push, onglet **Actions** de la PR : ouvrez **Chargement initial (lot 1)**, **Incrément (lot 2)**, **Contrôler l'incrément et enregistrer l'empreinte**, **Rejeu du lot 2 (ne doit rien changer)**, puis **Vérifier les résultats** ; la dernière ligne doit être `Résultat : 52 OK, 0 ÉCHEC.`

## Erreurs courantes

- CI rouge avec `ModuleNotFoundError` alors que tout passe chez vous : Codex a utilisé une bibliothèque absente de `requirements.txt`.
- Passer à `tout` trop tôt : la CI est rouge pour une raison attendue, et l'équipe s'habitue à ignorer le rouge.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M4 · Épisode 6 : le défaut planté

**Quoi :** un exercice de revue sur la PR `increment-scd2`. Quand tout est vert, **A** fait introduire par Codex **un** défaut, sans dire lequel à B. B doit le trouver, A le corrige, B relit encore. **Pourquoi :** c'est la seule façon de vérifier qu'une revue trouve quelque chose, et de voir la chaîne complète revue → correction → revue.

| Défaut (A en choisit un) | Ce qui peut arriver (selon le code de Codex) |
|---|---|
| 1 · `valid_to` traité comme **inclus** | `increment` peut planter (deux versions de NBE valables le 15 juillet) ; ou un ÉCHEC du type … `faits mal rattachés` |
| 2 · la zone est **écrasée** (UPDATE) au lieu d'ajouter une version | un ÉCHEC du type `version attendue absente (…)`, parfois après d'autres ÉCHEC ; ou un plantage si une vente ne trouve plus de version |
| 3 · les retours **ANNULE** sont chargés | un ÉCHEC du type `fact_retour : attendu 99 lignes, observé <n>` |

Un défaut produit **un** de trois symptômes : (a) `python -m solen increment` **plante** : traceback Python, la transaction annule le lot 2, puis beaucoup de lignes ÉCHEC ; (b) le vérificateur affiche un **ÉCHEC** ; (c) un **chiffre faux sans erreur**, visible seulement face à la référence. Les messages du tableau sont **possibles**, pas promis.

**Demande à Codex · texte à adapter**

```
Exercice de revue demandé par le professeur, sur la branche increment-scd2.
Dans solen/, introduis UN seul défaut : <1, 2 ou 3, recopié du livret>.
Ne touche ni à tests/, ni à controles/, ni aux sources.
Ne mets aucun commentaire qui signale le défaut dans le code.
Écris seulement en toute fin de réponse une ligne « NOTE AUTEUR : »
qui dit le fichier et la ligne modifiés. Ne fais ni commit ni push.
```

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git add solen/
git commit -m "Incrément : ajustements"
git push
```

Gardez la NOTE AUTEUR pour vous. Avec `ETAPE_CI: rejeu`, la CI de la PR devrait passer au rouge : c'est le premier indice de B.

## Règle du jeu

Le défaut est planté **pour l'exercice** et sera corrigé dans la même PR avant toute fusion. Jamais de défaut volontaire hors exercice annoncé.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M4 · Revue → correction → revue

## B · reproduire, lire, localiser, demander la correction

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git fetch
git switch increment-scd2
python -m solen initial
python -m solen increment
python -m controles.verifier --mission tronc-commun --etape increment
git show HEAD
```

**Demande à Codex · texte à adapter**

```
Utilise $revoir-bi. Branche : increment-scd2. Lecture seule : ne modifie
rien, ne lance aucun chargement. Angle : SCD2 et rejeu.
Rapporte au plus 3 défauts établis (fichier:ligne, règle violée, exemple
observable, correction proposée), puis les contrôles non réalisés.
Symptôme que j'ai reproduit : <dernière ligne du traceback, ou première
ligne ÉCHEC, ou chiffre différent de la référence>.
```

B **lit** le symptôme (dernière ligne du traceback et le fichier cité, première ligne ÉCHEC, ou écart avec la référence) et le **localise** dans `git show HEAD`, le dernier commit de A. Revue GitHub : **Request changes**, avec la commande, le symptôme, fichier et ligne, la règle et la correction demandée.

## A · corriger, puis B · deuxième revue

**Demande à Codex · texte à adapter**

```
Voici les constats de revue reçus sur ma PR <n> : <coller>.
Pour chacun : établi, non établi ou non vérifiable, avec une preuve.
Corrige seulement les défauts établis, dans solen/. Relance initial,
increment, --etape increment, increment, --etape rejeu et pytest -q ;
recopie les sorties. Ne fais ni commit ni push.
```

A commite et pousse la correction. B ouvre une **nouvelle** conversation : « Utilise $revoir-bi sur la version corrigée de increment-scd2 : chaque constat initial est-il corrigé, encore présent ou non vérifié ? Cherche une erreur introduite par la correction. Ne modifie rien. » CI verte + **Approve** : **B** fusionne ; puis les deux `git switch main`, `git pull`.

### Erreurs courantes

- Coller la conclusion de Codex comme revue GitHub : écrivez ce que **vous** avez reproduit.
- « Aucun défaut établi » n'est pas « tout est juste » : c'est le résultat d'une lecture, sous un angle.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M5 · Un conflit Git contrôlé

**Quoi :** provoquer volontairement un **conflit** : A et B modifient la **même ligne** de `notes/JOURNAL.md` dans deux branches. **Pourquoi :** un conflit n'est pas une panne ; Git demande simplement à un humain de choisir. Mieux vaut l'apprendre sur un journal que sur du code. Le professeur le montre d'abord (10 min) ; chaque binôme le reproduit ensuite (15 min).

| Qui | Geste |
|---|---|
| A et B | partent d'un `main` à jour : `git switch main`, `git pull` |
| A | `git switch -c journal-a` ; dans la dernière entrée du journal, réécrit la ligne **Prochain objectif** : « exporter les tables » (une phrase différente de celle déjà écrite) ; commit, push, PR |
| B | `git switch -c journal-b` ; réécrit **la même ligne** : « écrire un Skill de revue SCD2 » ; commit, push, PR |
| B | relit et fusionne la PR de A. La PR de B affiche alors **This branch has conflicts that must be resolved** |

## B · faire apparaître le conflit sur son poste

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
git switch main
git pull
git switch journal-b
git merge main
```

**Sortie attendue · ne pas recopier**

```
Auto-merging notes/JOURNAL.md
CONFLICT (content): Merge conflict in notes/JOURNAL.md
Automatic merge failed; fix conflicts and then commit the result.
```

**Fichier en conflit · notes/JOURNAL.md, extrait**

```
<<<<<<< HEAD
- **Prochain objectif** : écrire un Skill de revue SCD2 (preuve : PR relue).
=======
- **Prochain objectif** : exporter les tables (preuve : --etape export OK).
>>>>>>> main
```

Entre `<<<<<<<` et `=======` : votre version (`HEAD`, la branche où vous êtes). Entre `=======` et `>>>>>>>` : celle qui arrive, nommée après `>>>>>>>` (`main` ici, `origin/main` dans l'exemple du livret 01).

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M5 · Résoudre avec Codex, finir avec Git

Codex **propose** la résolution ; vous **décidez** et faites les gestes Git.

**Demande à Codex · texte à adapter**

```
J'ai un conflit Git. Voici la sortie exacte de git status :
<coller>
Explique en mots simples ce qui s'est passé (quelles deux versions se
rencontrent). Pour notes/JOURNAL.md, montre les deux blocs et dis ce que
chacun voulait garder. Propose une résolution qui garde l'intention des
deux. Ne modifie le fichier qu'après mon accord, et seulement lui.
Ne lance ni git add, ni git commit, ni git push : je les fais moi-même.
```

Résolution attendue ici : garder **les deux** objectifs, dans l'ordre que vous choisissez, sans aucun marqueur. Puis :

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
grep -n "<<<<<<<\|=======\|>>>>>>>" notes/JOURNAL.md
git add notes/JOURNAL.md
git commit --no-edit
git push
```

Le `grep` ne doit **rien** afficher. `git commit --no-edit` garde le message proposé par Git (`Merge branch 'main' into journal-b`). Sur GitHub, la PR de B redevient fusionnable : A relit et fusionne. Puis les deux : `git switch main`, `git pull`.

## Erreurs courantes

- Garder « sa » version d'un clic sans lire l'autre : le travail du binôme disparaît.
- Laisser un marqueur `=======` : dans un fichier Python, le code ne s'exécute plus ; dans le journal, la PR montre un texte cassé.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M6 · Extension facultative : un Skill local

**Facultatif** : si le temps le permet, ou en travail personnel ; la séance passe directement à S4-M7. Un **Skill** est une méthode réutilisable, écrite en français dans `.agents/skills/<nom>/SKILL.md`, avec un en-tête `name` et `description`. Il n'ajoute ni modèle, ni outil, ni quota : il évite de réécrire la même consigne à chaque fois. Vous en avez déjà utilisé trois.

| Skill fourni | Ce qu'il fait | Ce qu'il ne fait jamais |
|---|---|---|
| `$expliquer-bi` | explique une notion ou un code avec un exemple Solen chiffré, termine par une question | modifier un fichier |
| `$revoir-bi` | relit en lecture seule ; au plus 3 défauts établis ; contrôles non réalisés à part | dire « tout est correct » |
| `$preparer-pr` | liste OK / À corriger / Non vérifié ; titre et description de PR | commit, push, ouvrir la PR |

## Créer le vôtre : une règle, observable

**Demande à Codex · texte à adapter**

```
Crée .agents/skills/revue-scd2-zitouna/SKILL.md, en partant de
.agents/skills/revoir-bi/SKILL.md. En-tête : name: revue-scd2-zitouna,
et une description d'une phrase qui dit quand l'utiliser.
Garde la lecture seule et la limite de 3 défauts établis.
Ajoute cette règle précise : contrôler une vente NBE du 2026-07-14 et
une du 2026-07-15, et dire pour chacune la boutique_key et la zone
retenues. Ne modifie pas les Skills fournis. Montre le fichier complet.
```

**Essayer**, dans une nouvelle conversation : « Utilise $revue-scd2-zitouna pour relire l'historique sur main. Lecture seule. » La réponse doit citer les deux dates et deux zones différentes. Si le Skill n'est pas proposé : redémarrez l'application et rouvrez le dossier, ou écrivez « Lis .agents/skills/revue-scd2-zitouna/SKILL.md et applique sa méthode ». Notez quelle route a marché.

Le Skill est un fichier du dépôt : il passe par une branche (`skill-revue-scd2`) et une courte PR, comme le reste.

## Erreurs courantes

- Une règle vague (« vérifie bien tout ») : elle ne change rien à la réponse.
- Croire qu'un Skill remplace le vérificateur : comparez toujours avec `python -m controles.verifier`.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M7 · Export et livraison

**Quoi :** écrire chaque table dans `outputs/export/<table>.csv` et un `manifest.json` (comptes et sommes). **Pourquoi :** c'est la livraison du tronc commun : Power BI (séance 5) lira ces CSV, et le manifeste prouve que l'export est complet. Ici commence le dernier bloc (50 min), « Passer à votre mission ». **B** est auteur, branche `export-manifest`.

**Demande à Codex · texte à adapter**

```
Ajoute la commande python -m solen export : pour chaque table de
l'entrepôt, outputs/export/<table>.csv (UTF-8, séparateur virgule,
en-tête = noms de colonnes) et outputs/export/manifest.json au format
du cahier des charges § 10 (lignes et sommes par table).
Puis passe ETAPE_CI de rejeu à tout dans .github/workflows/controles.yml.
Lance export, python -m controles.verifier --mission tronc-commun
--etape export et pytest -q ; recopie les sorties.
J'attends 2345 lignes dans fact_vente.csv, en-tête non compté : compare.
Ne fais ni commit ni push.
```

**Terminal Ubuntu (bash) · dans ~/solen-bi, venv actif**

```
python -m solen export
python -m controles.verifier --mission tronc-commun --etape export
```

**Sortie attendue · ne pas recopier · extrait**

```
OK — export fact_vente.montant_brut_millimes : somme 95849090 (CSV = entrepôt
= manifest)
Résultat : 12 OK, 0 ÉCHEC.
```

| Table | Lignes | Sommes contrôlées |
|---|---|---|
| `fact_vente` | 2345 | quantite 4412 · montant_brut_millimes 95 849 090 |
| `fact_retour` | 99 | quantite 148 · montant_millimes 4 404 630 |
| `dim_produit` | 8 | prix_millimes 1 434 900 · cout_achat_millimes 908 100 |

## Erreurs courantes

• Retoucher un CSV exporté à la main : `observé CSV 2344 et manifest 2345`. On relance `export` ; on ne corrige jamais une sortie. `wc -l` affiche 2346 : l'en-tête compte pour une ligne.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# S4–M8 · Passer à votre mission

**Quoi :** une PR, à deux, qui fait passer le dépôt du tronc commun à votre mission, une fois la PR `export-manifest` fusionnée. **Pourquoi :** la séance 5 commence sur votre mission ; le code du tronc commun est gardé et **généralisé**, pas réécrit. **A** est auteur. Exemple pour `M03` : remplacez par votre code.

**Terminal Ubuntu (bash) · ~/solen-bi · gestes 1, 0, 2 et 3**

```
git switch main && git pull
git switch -c mission-M03
python -m solen initial && python -m solen export
mkdir -p livraison/tronc-commun/export-lot1
cp outputs/export/* livraison/tronc-commun/export-lot1/
python -m solen increment && python -m solen export
mkdir -p livraison/tronc-commun/export-lot2
cp outputs/export/* livraison/tronc-commun/export-lot2/
git add livraison && git commit -m "Livraison du tronc commun (lots 1 et 2)"
rm -rf outputs/*
echo "M03" > mission.txt
```

**Geste 0, la livraison :** deux exports du tronc commun, après le lot 1 puis après le lot 2, copiés dans `livraison/tronc-commun/export-lot1/` et `export-lot2/` et commités : de petits CSV, la preuve livrée, que Power BI lira en séance 5. **Geste 2 :** `rm -rf outputs/*` vide `outputs/`, qui appartient à la mission courante ; le fichier caché `.gitkeep` reste (`*` ne prend pas les fichiers cachés) : vérifiez avec `ls -a outputs`. Oublier ce geste laisse de faux ÉCHEC du tronc commun. **Geste 3 :** `mission.txt` contient votre code.

## Geste 4 : la première demande à Codex, puis la PR

Livret 04, « Passer à votre mission (2/2) » : Codex **généralise** `solen/` pour votre mission, première étape seulement : les rejets du lot 1. Contrôle : `--etape rejets` → 0 ÉCHEC ; les tests de `tests/` qui lisent `mission.txt` passent ou sont adaptés ; `ETAPE_CI` revient à `aucune` dans la même PR. Puis `git push`, PR, revue de B, fusion.

02 / TRONC COMMUN · SÉANCE 4 · LE BESOIN CHANGE

# Fin de séance 4 · Bilan et suite

| Preuve | Où la voir |
|---|---|
| `--etape increment` 51 OK puis `--etape rejeu` 52 OK, 0 ÉCHEC | terminal |
| dim_boutique 4 versions ; NBE le 14/07 → Cap Bon, le 15/07 → Nord-Est | vérificateur |
| Journal des chargements : 3 entrées, le rejeu à 0 ajout | `outputs/journal_chargements.json` |
| Défaut planté trouvé par B, corrigé par A, relu, fusionné | PR increment-scd2 |
| Conflit résolu, aucun marqueur | PR journal-b, `grep` |
| `--etape export` 12 OK ; livraison commitée (lots 1 et 2) | `livraison/tronc-commun/` |
| Première PR de la mission : `mission.txt` = M0x, `--etape rejets` 0 ÉCHEC | PR mission-M0x |

### À ce stade, je sais…

[ ] expliquer SCD2 avec la vente du 14 et celle du 15 juillet ;
[ ] dire pourquoi relancer `increment` n'ajoute rien, et le prouver ;
[ ] lire un test pytest et dire quelle règle il protège ;
[ ] reproduire un défaut, le localiser dans un diff et demander sa correction ;
[ ] résoudre un conflit et finir les gestes Git soi-même ;
[ ] dire pourquoi `outputs/` est vidé et `livraison/` commitée ;
[ ] (facultatif) créer un Skill avec une règle observable.

## La suite : séance 5 et travail personnel

• **Séance 5, première moitié** : votre mission, `initial` → `increment` → `rejeu` → `export` → app (son écran « Qualité des données » lit le journal des chargements), un cran vert par PR, revue croisée. Livret 04 et guide de votre mission.

• **Séance 5, seconde moitié** : Power BI Desktop sur le tronc commun depuis `livraison/tronc-commun/` (actualisation : lot 1, puis lot 2), lecture des visuels, répétition. Livret 05.

### Dernier geste de la séance

Une note d'état (fiche `prompts/16-reprendre.md`), vérifiée contre le terminal, puis l'entrée du journal de la dernière PR : état, décision, preuve réelle, prochain objectif.

02 / TRONC COMMUN · ANNEXES

# Annexe · Les nombres de référence

Source : `tronc-commun/reference/README.md`, `attendu_lot1.json`, `attendu_lot2.json` et `echantillon/attendu.md`. Calculés par le générateur des données, pas par une solution. Montants stockés en millimes, affichés ici en TND.

| | Échantillon | Après lot 1 (`initial`) | Après lot 2 (= rejeu) |
|---|---|---|---|
| Lignes de `ventes.csv` lues | 12 | 1998 | + 413 |
| Doublons exacts ignorés | 1 | 14 | + 4 |
| Ventes déjà chargées (ignorées) | — | 0 | + 15 |
| Rejets `ventes.csv` | 5 | 24 | + 9 |
| Rejets `operations.sqlite` | 1 | 2 | + 1 |
| Rejets : cle_inconnue / id_conflit / id_manquant / quantite_invalide | 1 / 2 / 1 / 1 | 8 / 6 / 5 / 7 (26) | lot2 : 4 / 2 / 2 / 2 (10) |
| `fact_vente` / `fact_retour` | 6 / 1 | 1960 / 76 | 2345 / 99 |
| `dim_date` / `dim_produit` / `dim_boutique` | 31 / 8 / 3 | 181 / 8 / 3 | 243 / 8 / 4 |
| Ventes brutes | 127,350 | 78 067,830 | 95 849,090 |
| Retours | 14,500 | 3 758,330 | 4 404,630 |
| Ventes nettes | 112,850 | 74 309,500 | 91 444,460 |
| Coût d'achat des unités nettes | 67,250 | 45 317,800 | 55 837,600 |
| Marge sur coût d'achat | 45,600 | 28 991,700 | 35 606,860 |
| Taux de retour | 11,39 % | 4,81 % | 4,60 % |
| Unités nettes | — | 3605 | 4264 |

Contrôles : `--etape rejets` → 5 OK ; `--etape initial` → `Résultat : 51 OK, 0 ÉCHEC.` ; `--etape increment` → 51 OK, puis `--etape rejeu` → 52 OK ; `--etape export` → 12 OK. SCD2 : NBE le 2026-06-30 et le 2026-07-14 → Cap Bon ; le 2026-07-15 → Nord-Est.

02 / TRONC COMMUN · ANNEXES

# Annexe · Lire un ÉCHEC et réagir

Lisez la **première** ligne ÉCHEC : les suivantes en découlent souvent. Puis la fiche `prompts/10-erreur.md` : message exact collé, votre hypothèse, diagnostic **avant** toute correction. Ne modifiez jamais le vérificateur ni la référence.

| Le message contient… | Cause probable | Premier geste |
|---|---|---|
| `entrepôt absent` | pipeline pas lancé, ou mauvais dossier | `pwd`, puis `python -m solen initial` |
| `table … absente` ou `colonnes absentes` | noms différents du cahier des charges | demander de respecter le § 6 mot pour mot |
| `attendu N lignes, observé M` | doublons gardés, rejet oublié, lot rechargé | comparer M − N aux comptes de l'annexe précédente |
| `clé métier … en double` | insertion non idempotente | tester la clé métier avant d'insérer |
| `attendu des entiers` | montants stockés en décimaux | exiger des millimes entiers (`int`) |
| `ligne orpheline` ou `foreign_key_check` | faits chargés avant les dimensions, ou mauvaise clé | vérifier l'ordre de chargement |
| `version attendue absente` | SCD2 écrasé ou dates de validité fausses | relire le § 7 ; jamais d'`UPDATE` de la zone |
| `faits mal rattachés` | `valid_to` inclus, ou jointure par `boutique_id` | relire la requête qui choisit la version |
| `rejets lot… : <motif>, attendu, observé` | ordre des règles ; retours oubliés | suivre une ligne rejetée avec `$expliquer-bi` |
| `rejeu : … changements dans [...], empreinte absente` | la relance modifie l'entrepôt ; `--etape increment` oublié | comparer avant / après ; ordre : increment, relance, rejeu |
| `export … CSV … manifest` | export obsolète ou retouché | relancer `python -m solen export` |
| `ne correspond à aucun lot de référence` | état intermédiaire (`--etape tout`) | lancer `--etape initial` ou `--etape increment` pour le détail |
| `ÉCHEC — mission … inconnue` (code 2) | `mission.txt` vide ou mal écrit | `cat mission.txt` |